Workshop guidé — Terraform + Ansible : StockLine de bout en bout

> **Bloc 11 CL-IAC** · Durée : **1 journée (7 h)** · Environnement : votre WSL `lab-m2i` (Ubuntu 24.04) avec Docker, Terraform et Ansible · Coût : **0 €** (tout tourne sur votre poste)

Objectifs

À la fin de la journée, vous saurez :

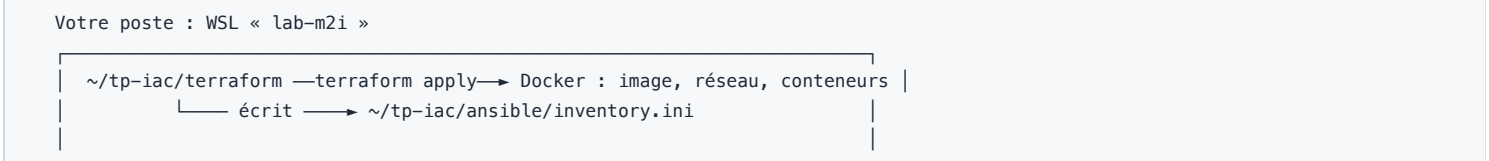
- construire avec **Terraform** (provider Docker) un parc de serveurs Linux à partir d'une image que vous fabriquez vous-même ;
- faire **écrire l'inventaire Ansible par Terraform** : plus aucune adresse ni aucun port recopié à la main ;
- configurer ces serveurs avec **Ansible organisé en rôles** : templates Jinja2, handlers, variables, coffre chiffré (vault) ;
- déployer **StockLine** (API FastAPI + PostgreSQL) derrière un répartiteur de charge nginx, et le vérifier de bout en bout ;
- **passer à l'échelle** (un 3e serveur d'API) en modifiant une seule ligne de code, puis **tout détruire** proprement.

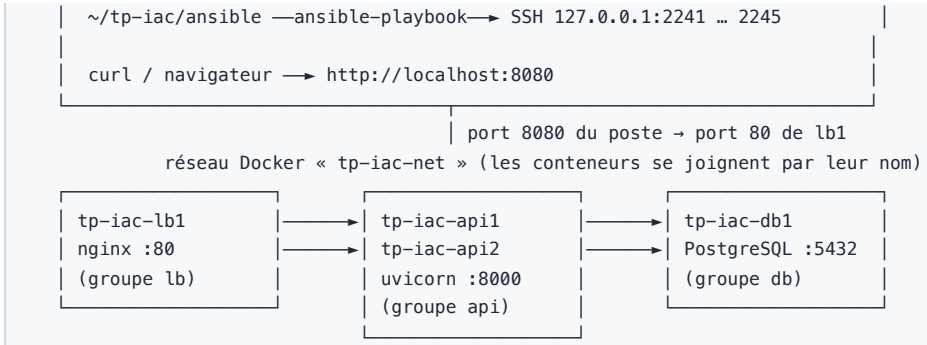
Mise en situation

L'équipe StockLine en a assez des environnements montés « à la main » : personne ne sait plus ce qui est installé où, et chaque nouveau serveur prend une demi-journée. Votre mission : livrer un environnement complet, **décrit entièrement par du code**, qui se crée en deux commandes, se modifie en changeant le code, et se détruit sans laisser de trace.

Vous travaillez en local, sur Docker : chaque « serveur » est un conteneur Ubuntu qui fait tourner systemd et un serveur SSH, exactement comme les conteneurs `lab-web1`, `lab-web2` et `lab-db1` de l'exercice Ansible du cours. Ansible ne fait aucune différence avec une vraie machine.

Architecture cible





Outil	Ce qu'il possède
Terraform	Ce qui existe : l'image, le réseau, les conteneurs, et le fichier d'inventaire
Ansible	Ce qu'il y a **dans** les serveurs : paquets, base de données, application, nginx

Règles du workshop

1. **Un seul terminal** : Ubuntu WSL (`lab-m2i`). Pas PowerShell, pas Git Bash.
2. Les blocs **`bash`** se copient-collent **en entier**, **dans l'ordre**, avec le bouton **Copier** de la version HTML de cet énoncé (ouverte dans votre navigateur Windows), puis un clic droit ou `Ctrl+Maj+V` dans le terminal Ubuntu. **Ne copiez jamais depuis le PDF** : il coupe les longues lignes et peut casser l'indentation des fichiers. Chaque bloc commence par un `cd` : vous pouvez fermer votre terminal entre deux blocs sans rien perdre.
3. Les blocs **`text`** montrent la sortie attendue. Les identifiants, adresses IP, durées et numéros de version peuvent différer chez vous.
4. Les vérifications affichent **`OK`** ou **`ERREUR`**. Sur une `ERREUR` : appliquez la fiche de la section Dépannage indiquée, sinon appelez le formateur. **Ne passez pas à la suite avec une ERREUR.**
5. Un bloc qui commence par `cat > fichier <<'EOF'` **crée (ou remplace) le fichier en entier**. Quand un fichier évolue au cours de la journée, on le recolle en entier : c'est voulu.
6. Avant de coller un fichier, **lisez l'explication qui le précède** : le but est de comprendre chaque ligne, pas de copier vite.
7. **Tous les fichiers du sujet sont aussi fournis** dans le dossier `ressources/` du ZIP, pour les relire dans VS Code ou vérifier une saisie : `ressources/partie-N/` contient chaque fichier créé ou modifié dans la partie N, dans sa version exacte de cette partie et rangé comme dans `~/tp-iac` . La méthode de travail reste le copier-coller des blocs : ce sont eux qui ont été testés.

Planning indicatif

Horaire	Partie
9h00 - 9h20	0. Vérifier l'environnement

Horaire	Partie
9h20 - 9h45	1. Préparer le projet
9h45 - 10h30	2. Terraform : l'image cible et le réseau
10h45 - 11h45	3. Terraform : un module « serveur » et le parc
11h45 - 12h30	4. Terraform écrit l'inventaire Ansible
13h30 - 14h15	5. Ansible : configuration du projet et rôle « commun »
14h15 - 15h00	6. Coffre chiffré et rôle « postgresql »
15h15 - 16h00	7. Rôle « stockline » : l'API
16h00 - 16h40	8. Rôle « nginx » et vérification de bout en bout
16h40 - 17h15	9. Passer à l'échelle : un 3e serveur d'API
17h15 - 17h30	10. Tout détruire

Partie 0 — Vérifier l'environnement (20 min)

On vérifie que les outils installés en début de formation (fichier `00-WSL/commands.sh`, copié dans `ressources/cours/` du ZIP) sont là et que rien ne bloquera la journée : Docker utilisable **sans sudo**, ports libres, dossier de travail inexistant.

```
cd ~
echo "--- Versions installées ---"
docker --version
terraform -version | head -n 1
ansible --version | head -n 1
echo "--- Contrôles ---"
if docker ps > /dev/null 2>&1; then echo "OK : Docker répond sans sudo"; else echo "ERREUR : Docker ne répond pas sans sudo (fiche
        D1)"; fi
if command -v terraform > /dev/null; then echo "OK : Terraform est installé"; else echo "ERREUR : Terraform est absent (fiche D
        2)"; fi
if command -v ansible-playbook > /dev/null; then echo "OK : Ansible est installé"; else echo "ERREUR : Ansible est absent (fiche D
        2)"; fi
if ansible-galaxy collection list 2> /dev/null | grep -q "community.postgresql"; then echo "OK : collection community.postgresql p
        résente"; else echo "ERREUR : collection community.postgresql absente (fiche D3)"; fi
for port in 2241 2242 2243 2244 2245 8080; do
  if ss -ltn | grep -q ":$port "; then echo "ERREUR : le port $port est déjà utilisé (fiche D4)"; else echo "OK : port $port libr
        e"; fi
done
if [ -e ~/tp-iac ]; then echo "ERREUR : le dossier ~/tp-iac existe déjà (fiche D5)"; else echo "OK : le dossier ~/tp-iac n'existe
        pas encore"; fi
```

Sortie attendue (versions indicatives) :

```
--- Versions installées ---
Docker version 29.9.2, build a82e528
Terraform v1.17.0
ansible [core 2.21.5]
--- Contrôles ---
OK : Docker répond sans sudo
OK : Terraform est installé
OK : Ansible est installé
OK : collection community.postgresql présente
OK : port 2241 libre
OK : port 2242 libre
OK : port 2243 libre
OK : port 2244 libre
OK : port 2245 libre
OK : port 8080 libre
OK : le dossier ~/tp-iac n'existe pas encore
```

✅ **Point de contrôle 0** : toutes les lignes de contrôle commencent par `OK`.

Partie 1 — Préparer le projet (25 min)

1.1 Une clé SSH dédiée au workshop

Ansible se connecte aux serveurs en SSH, **avec une clé**, jamais avec un mot de passe. On crée une paire de clés réservée à ce workshop :

- `~/.ssh/tp_iac_ed25519` : la clé **privée**. Elle ne quitte jamais votre poste ;
- `~/.ssh/tp_iac_ed25519.pub` : la clé **publique**. C'est elle qu'on dépose sur les serveurs.

La commande ne crée la clé que si elle n'existe pas déjà (`test -f … ||`).

```
cd ~
mkdir -p ~/.ssh
chmod 700 ~/.ssh
test -f ~/.ssh/tp_iac_ed25519 || ssh-keygen -t ed25519 -N "" -C "tp-iac" -f ~/.ssh/tp_iac_ed25519
ls -l ~/.ssh/tp_iac_ed25519 ~/.ssh/tp_iac_ed25519.pub
```

`ssh-keygen` affiche d'abord une empreinte (`SHA256:…`) et un petit dessin (« randomart ») : c'est normal. Les deux dernières lignes doivent ressembler à :

```
-rw------- 1 vous vous 399 oct. 27 09:25 /home/vous/.ssh/tp_iac_ed25519
-rw-r--r-- 1 vous vous  88 oct. 27 09:25 /home/vous/.ssh/tp_iac_ed25519.pub
```

1.2 L'arborescence du projet

Tout le workshop vit dans `~/tp-iac`, **dans le système de fichiers Linux de WSL** (jamais sous `/mnt/c/…` : Ansible y refuse son fichier de configuration).

```
cd ~
mkdir -p ~/tp-iac/image-cible ~/tp-iac/stockline/app ~/tp-iac/terraform/templates ~/tp-iac/terraform/modules/serveur ~/tp-iac/ansible/group_vars/all
cp ~/.ssh/tp_iac_ed25519.pub ~/tp-iac/image-cible/authorized_keys
ls ~/tp-iac
```

```
ansible
image-cible
stockline
terraform
```

> ⚠️ On copie la clé **publique** (`.pub`) dans `authorized_keys`, jamais la clé privée.

Le fichier `.gitignore` liste ce qui ne doit **jamais** être commité : le state Terraform (il peut contenir des données sensibles), le mot de passe du coffre Ansible et l'inventaire généré.

`~/tp-iac/.gitignore`

```
cd ~/tp-iac
cat > .gitignore <<'EOF'
# Terraform : dossier des providers, state (contient des données sensibles), plans
terraform/.terraform/
terraform/*.tfstate
terraform/*.tfstate.*
terraform/tfplan*

# Ansible : mot de passe du coffre (JAMAIS commité) et inventaire généré
ansible/.vault-pass
ansible/inventory.ini
EOF
```

1.3 Le code de l'application StockLine

L'application fil rouge est fournie : vous n'avez pas à la modifier. Ses 4 fichiers Python et sa liste de dépendances sont créés par les blocs ci-dessous ; ils se trouvent aussi dans le dossier `ressources/stockline/` du ZIP. C'est Ansible qui la copiera plus tard sur les serveurs d'API.

`~/tp-iac/stockline/requirements.txt`

```
cd ~/tp-iac
cat > stockline/requirements.txt <<'EOF'
fastapi==0.115.6
uvicorn[standard]==0.34.0
sqlalchemy==2.0.36
pydantic==2.10.4
httpx==0.28.1
pytest==8.3.4
# Pilote PostgreSQL — utilisé à partir de CL-TP1 quand DATABASE_URL pointe vers PostgreSQL.
# Décommentez la ligne suivante lorsque vous ciblez PostgreSQL :
# psycopg2-binary==2.9.10
EOF
```

~/tp-iac/stockline/app/__init__.py

```
cd ~/tp-iac
cat > stockline/app/__init__.py <<'EOF'
"""StockLine — application fil rouge du cursus Administrateur Cloud 2026."""
EOF
```

~/tp-iac/stockline/app/db.py

```
cd ~/tp-iac
cat > stockline/app/db.py <<'EOF'
"""StockLine — couche base de données (SQLAlchemy).

Par défaut, l'application utilise une base SQLite locale (fichier
``stockline.db`` dans le répertoire courant) : aucune configuration
n'est nécessaire pour démarrer.

Pour utiliser PostgreSQL (blocs suivants du cursus), définissez la
variable d'environnement DATABASE_URL, par exemple :

    export DATABASE_URL="postgresql+psycopg2://stockline:motdepasse@localhost:5432/stockline"
"""

import os
from datetime import datetime, timezone

from sqlalchemy import (
    Column,
    DateTime,
    Float,
    ForeignKey,
    Integer,
    String,
    create_engine,
)
from sqlalchemy.orm import Session, declarative_base, sessionmaker

# 1. URL de connexion : PostgreSQL si DATABASE_URL est définie, sinon SQLite.
DATABASE_URL = os.getenv("DATABASE_URL", "sqlite:///./stockline.db")

# SQLite impose par défaut un accès mono-thread : on assouplit ce réglage
# car FastAPI peut traiter les requêtes dans plusieurs threads.
connect_args = {"check_same_thread": False} if DATABASE_URL.startswith("sqlite") else {}
```

```
engine = create_engine(DATABASE_URL, connect_args=connect_args)
SessionLocal = sessionmaker(bind=engine, autocommit=False, autoflush=False)

Base = declarative_base()


def maintenant_utc() -> datetime:
    """Horodatage UTC (les serveurs travaillent toujours en UTC)."""
    return datetime.now(timezone.utc)


class ProduitDB(Base):
    """Table des produits référencés dans l'inventaire."""

    __tablename__ = "produits"

    id = Column(Integer, primary_key=True, index=True)
    reference = Column(String(50), unique=True, index=True, nullable=False)
    nom = Column(String(200), nullable=False)
    prix_unitaire = Column(Float, nullable=False)
    seuil_alerte = Column(Integer, nullable=False, default=5)
    cree_le = Column(DateTime, default=maintenant_utc, nullable=False)


class MouvementDB(Base):
    """Table des mouvements de stock (entrées et sorties)."""

    __tablename__ = "mouvements"

    id = Column(Integer, primary_key=True, index=True)
    produit_id = Column(Integer, ForeignKey("produits.id"), nullable=False, index=True)
    type = Column(String(10), nullable=False)  # "entree" ou "sortie"
    quantite = Column(Integer, nullable=False)
    commentaire = Column(String(300), nullable=True)
    cree_le = Column(DateTime, default=maintenant_utc, nullable=False)


def init_db() -> None:
    """Crée les tables si elles n'existent pas (appelé au démarrage)."""
    Base.metadata.create_all(bind=engine)


def get_session():
    """Fournit une session de base de données à chaque requête (dépendance FastAPI)."""
    session: Session = SessionLocal()
    try:
        yield session
    finally:
        session.close()
EOF
```

~/tp-iac/stockline/app/models.py

```
cd ~/tp-iac
cat > stockline/app/models.py <<'EOF'
"""StockLine — schémas Pydantic (validation des entrées/sorties de l'API).

Deux familles de modèles :
- les modèles ``...Create`` : ce que le client envoie (POST) ;
- les modèles de lecture : ce que l'API renvoie (avec id et horodatage).
"""
```

```python
from datetime import datetime
from enum import Enum

from pydantic import BaseModel, ConfigDict, Field


class TypeMouvement(str, Enum):
    """Un mouvement de stock est soit une entrée, soit une sortie."""

    ENTREE = "entree"
    SORTIE = "sortie"


# ---------------------------------------------------------------------------
# Produits
# ---------------------------------------------------------------------------

class ProduitCreate(BaseModel):
    """Données attendues pour créer un produit."""

    reference: str = Field(min_length=1, max_length=50, examples=["SSD-500"])
    nom: str = Field(min_length=1, max_length=200, examples=["Disque SSD 500 Go"])
    prix_unitaire: float = Field(gt=0, examples=[59.90])
    seuil_alerte: int = Field(default=5, ge=0, description="Stock minimal avant alerte")


class Produit(ProduitCreate):
    """Produit tel que renvoyé par l'API (avec identifiant)."""

    model_config = ConfigDict(from_attributes=True)

    id: int
    cree_le: datetime


# ---------------------------------------------------------------------------
# Mouvements
# ---------------------------------------------------------------------------

class MouvementCreate(BaseModel):
    """Données attendues pour enregistrer un mouvement de stock."""

    produit_id: int = Field(gt=0)
    type: TypeMouvement
    quantite: int = Field(gt=0, examples=[10])
    commentaire: str | None = Field(default=None, max_length=300)


class Mouvement(MouvementCreate):
    """Mouvement tel que renvoyé par l'API."""

    model_config = ConfigDict(from_attributes=True)

    id: int
    cree_le: datetime


# ---------------------------------------------------------------------------
# Stocks et santé
# ---------------------------------------------------------------------------

class Stock(BaseModel):
    """État du stock d'un produit (calculé à partir des mouvements)."""

    produit_id: int
    reference: str
    nom: str
```

```
    quantite: int
    seuil_alerte: int
    alerte: bool = Field(description="True si la quantité est sous le seuil d'alerte")


class Sante(BaseModel):
    """Réponse du health check GET /sante."""

    statut: str
    base_de_donnees: str
    version: str
EOF
```

~/tp-iac/stockline/app/main.py

```
cd ~/tp-iac
cat > stockline/app/main.py <<'EOF'
"""StockLine v1 — API de gestion d'inventaire (produits, mouvements, stocks).

Application fil rouge du cursus Administrateur Cloud 2026.

Lancement en local :
    uvicorn app.main:app --reload

Documentation interactive : http://127.0.0.1:8000/docs
"""

from contextlib import asynccontextmanager

from fastapi import Depends, FastAPI, HTTPException, status
from fastapi.middleware.cors import CORSMiddleware
from sqlalchemy import func, select, text
from sqlalchemy.orm import Session

from app import models
from app.db import MouvementDB, ProduitDB, get_session, init_db

VERSION = "1.0.0"


@asynccontextmanager
async def lifespan(app: FastAPI):
    """Au démarrage : création des tables si nécessaire."""
    init_db()
    yield


app = FastAPI(
    title="StockLine",
    description="API d'inventaire — fil rouge du cursus Administrateur Cloud",
    version=VERSION,
    lifespan=lifespan,
)

# Le front statique (front/index.html) est servi depuis une autre origine :
# on autorise les appels cross-origin (CORS). En production, restreindre la liste.
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_methods=["*"],
    allow_headers=["*"],
)
```

```python
# ---------------------------------------------------------------------------
# Santé
# ---------------------------------------------------------------------------

@app.get("/sante", response_model=models.Sante, tags=["exploitation"])
def sante(session: Session = Depends(get_session)) -> models.Sante:
    """Health check : l'API répond-elle et la base est-elle joignable ?"""
    try:
        session.execute(text("SELECT 1"))
        etat_bdd = "ok"
    except Exception:  # noqa: BLE001 — on veut un diagnostic, pas un crash
        etat_bdd = "indisponible"
    return models.Sante(statut="ok", base_de_donnees=etat_bdd, version=VERSION)


# ---------------------------------------------------------------------------
# Produits
# ---------------------------------------------------------------------------

@app.get("/produits", response_model=list[models.Produit], tags=["produits"])
def lister_produits(session: Session = Depends(get_session)) -> list[ProduitDB]:
    """Liste tous les produits référencés."""
    return list(session.execute(select(ProduitDB).order_by(ProduitDB.id)).scalars())


@app.post(
    "/produits",
    response_model=models.Produit,
    status_code=status.HTTP_201_CREATED,
    tags=["produits"],
)
def creer_produit(
    produit: models.ProduitCreate, session: Session = Depends(get_session)
) -> ProduitDB:
    """Crée un produit. La référence doit être unique."""
    existant = session.execute(
        select(ProduitDB).where(ProduitDB.reference == produit.reference)
    ).scalar_one_or_none()
    if existant is not None:
        raise HTTPException(
            status_code=status.HTTP_409_CONFLICT,
            detail=f"La référence '{produit.reference}' existe déjà (id={existant.id}).",
        )
    nouveau = ProduitDB(**produit.model_dump())
    session.add(nouveau)
    session.commit()
    session.refresh(nouveau)
    return nouveau


@app.get("/produits/{produit_id}", response_model=models.Produit, tags=["produits"])
def lire_produit(produit_id: int, session: Session = Depends(get_session)) -> ProduitDB:
    """Renvoie un produit par son identifiant."""
    produit = session.get(ProduitDB, produit_id)
    if produit is None:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail=f"Produit {produit_id} introuvable.",
        )
    return produit


# ---------------------------------------------------------------------------
# Mouvements
# ---------------------------------------------------------------------------
```

```python
def _quantite_en_stock(session: Session, produit_id: int) -> int:
    """Calcule le stock courant : somme des entrées moins somme des sorties."""
    total = 0
    lignes = session.execute(
        select(MouvementDB.type, func.sum(MouvementDB.quantite))
        .where(MouvementDB.produit_id == produit_id)
        .group_by(MouvementDB.type)
    ).all()
    for type_mouvement, somme in lignes:
        if type_mouvement == models.TypeMouvement.ENTREE.value:
            total += int(somme)
        else:
            total -= int(somme)
    return total


@app.get("/mouvements", response_model=list[models.Mouvement], tags=["mouvements"])
def lister_mouvements(session: Session = Depends(get_session)) -> list[MouvementDB]:
    """Liste tous les mouvements de stock (entrées et sorties)."""
    return list(session.execute(select(MouvementDB).order_by(MouvementDB.id)).scalars())


@app.post(
    "/mouvements",
    response_model=models.Mouvement,
    status_code=status.HTTP_201_CREATED,
    tags=["mouvements"],
)
def creer_mouvement(
    mouvement: models.MouvementCreate, session: Session = Depends(get_session)
) -> MouvementDB:
    """Enregistre un mouvement. Une sortie ne peut pas dépasser le stock courant."""
    produit = session.get(ProduitDB, mouvement.produit_id)
    if produit is None:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail=f"Produit {mouvement.produit_id} introuvable.",
        )
    if mouvement.type == models.TypeMouvement.SORTIE:
        stock = _quantite_en_stock(session, mouvement.produit_id)
        if mouvement.quantite > stock:
            raise HTTPException(
                status_code=status.HTTP_400_BAD_REQUEST,
                detail=(
                    f"Stock insuffisant pour '{produit.nom}' : "
                    f"{stock} en stock, sortie demandée de {mouvement.quantite}."
                ),
            )
    nouveau = MouvementDB(
        produit_id=mouvement.produit_id,
        type=mouvement.type.value,
        quantite=mouvement.quantite,
        commentaire=mouvement.commentaire,
    )
    session.add(nouveau)
    session.commit()
    session.refresh(nouveau)
    return nouveau


# ---------------------------------------------------------------------------
# Stocks
# ---------------------------------------------------------------------------

@app.get("/stocks/{produit_id}", response_model=models.Stock, tags=["stocks"])
def lire_stock(produit_id: int, session: Session = Depends(get_session)) -> models.Stock:
    """Renvoie l'état du stock d'un produit, avec indicateur d'alerte."""
```

```
    produit = session.get(ProduitDB, produit_id)
    if produit is None:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail=f"Produit {produit_id} introuvable.",
        )
    quantite = _quantite_en_stock(session, produit_id)
    return models.Stock(
        produit_id=produit.id,
        reference=produit.reference,
        nom=produit.nom,
        quantite=quantite,
        seuil_alerte=produit.seuil_alerte,
        alerte=quantite < produit.seuil_alerte,
    )
EOF
```

✅ **Point de contrôle 1** :

```
cd ~/tp-iac
for f in .gitignore image-cible/authorized_keys stockline/requirements.txt stockline/app/__init__.py stockline/app/db.py stocklin
        e/app/models.py stockline/app/main.py; do
  if [ -s "$f" ]; then echo "OK : $f"; else echo "ERREUR : $f manquant ou vide (recollez le bloc correspondant)"; fi
done
if python3 -c "import ast, sys; [ast.parse(open(f).read(), f) for f in sys.argv[1:]]" stockline/app/*.py; then echo "OK : le code
        Python est complet (aucune erreur de syntaxe)"; else echo "ERREUR : un fichier Python est incomplet (recollez son bloc)";
        fi
```

```
OK : .gitignore
OK : image-cible/authorized_keys
OK : stockline/requirements.txt
OK : stockline/app/__init__.py
OK : stockline/app/db.py
OK : stockline/app/models.py
OK : stockline/app/main.py
OK : le code Python est complet (aucune erreur de syntaxe)
```

Partie 2 — Terraform : l'image cible et le réseau (45 min)

2.1 Le Dockerfile de l'image cible

Chaque « serveur » de la journée sera un conteneur lancé à partir de cette image. Elle reprend celle de l'exercice Ansible du cours (`10-IaC/setup_ansible/docker/Dockerfile` , copié dans `ressources/cours/` du ZIP) :

Instruction	Rôle
`systemd systemd-sysv` + `CMD ["/sbin/init"]`	systemd tourne en PID 1, comme sur une vraie VM : on pourra gérer des services (`postgresql` , `nginx` , `stockline`)
`openssh-server`	le serveur SSH par lequel Ansible entre

Instruction	Rôle
`python3`	Ansible exécute ses modules en Python **sur la cible**
utilisateur `ansible` + `sudoers`	l'utilisateur de connexion, avec `sudo` sans mot de passe (pour `become: true`)
`COPY authorized_keys`	votre clé publique : seule votre clé privée ouvrira la porte

`~/tp-iac/image-cible/Dockerfile`

```
cd ~/tp-iac
cat > image-cible/Dockerfile <<'EOF'
# Image « cible » : un mini-serveur Ubuntu 24.04 administrable par Ansible.
# - systemd tourne en PID 1 (comme sur une vraie VM) : on pourra gérer des services ;
# - un serveur SSH écoute sur le port 22 ;
# - l'utilisateur « ansible » se connecte avec VOTRE clé publique et a sudo sans mot de passe ;
# - python3 est présent : Ansible en a besoin sur la machine cible.
FROM ubuntu:24.04

ENV DEBIAN_FRONTEND=noninteractive

RUN apt-get update \
 && apt-get install -y --no-install-recommends \
      systemd systemd-sysv openssh-server python3 sudo \
 && rm -rf /var/lib/apt/lists/*

RUN useradd -m -s /bin/bash ansible \
 && echo "ansible ALL=(ALL) NOPASSWD:ALL" > /etc/sudoers.d/ansible \
 && chmod 0440 /etc/sudoers.d/ansible \
 && mkdir -p /home/ansible/.ssh

COPY authorized_keys /home/ansible/.ssh/authorized_keys

RUN chown -R ansible:ansible /home/ansible/.ssh \
 && chmod 700 /home/ansible/.ssh \
 && chmod 600 /home/ansible/.ssh/authorized_keys

EXPOSE 22
# Signal d'arrêt propre de systemd (sinon « docker stop » attend 10 s puis tue le conteneur)
STOPSIGNAL SIGRTMIN+3
CMD ["/sbin/init"]
EOF
```

2.2 Les fichiers Terraform

`providers.tf` épingle les versions : la même version du provider Docker que dans les exercices du cours (`4.6.0`), et le provider `local` qui servira en partie 4 à écrire un fichier.

`~/tp-iac/terraform/providers.tf`

```
cd ~/tp-iac
cat > terraform/providers.tf <<'EOF'
# providers.tf — versions de Terraform et des providers, épinglées
```

```
terraform {
  required_version = ">= 1.5.0"

  required_providers {
    # Pilote le Docker Engine de votre WSL (images, réseaux, conteneurs)
    docker = {
      source  = "kreuzwerker/docker"
      version = "4.6.0"
    }
    # Écrit des fichiers sur votre poste (ici : l'inventaire Ansible)
    local = {
      source  = "hashicorp/local"
      version = "2.9.1"
    }
  }
}

# Sans argument : le provider parle au Docker local (socket /var/run/docker.sock)
provider "docker" {}
EOF
```

`variables.tf` ne contient pour l'instant qu'un préfixe, utilisé dans tous les noms Docker : on reconnaîtra au premier coup d'œil ce qui appartient au workshop.

`~/tp-iac/terraform/variables.tf`

```
cd ~/tp-iac
cat > terraform/variables.tf <<'EOF'
# variables.tf — les paramètres du projet
variable "prefixe" {
  type        = string
  description = "Préfixe de tous les noms Docker créés par ce projet"
  default     = "tp-iac"
}
EOF
```

`main.tf` décrit deux ressources :

- **`docker_image.cible`** construit l'image depuis le dossier `image-cible`. Le bloc `triggers` contient l'empreinte (`filesha1`) du Dockerfile et de la clé : si l'un des deux change, Terraform reconstruit l'image. Sans lui, Terraform ne regarderait que le nom de l'image et ne verrait jamais vos modifications ;
- **`docker_network.reseau`** crée un réseau privé. Sur un réseau créé ainsi, Docker fournit un **DNS** : les conteneurs se joindront par leur **nom** (`tp-iac-db1`), sans jamais connaître leurs adresses IP.

`path.module` désigne le dossier du fichier `.tf` : les chemins restent justes quel que soit l'endroit d'où vous lancez Terraform.

`~/tp-iac/terraform/main.tf`

```
cd ~/tp-iac
cat > terraform/main.tf <<'EOF'
# main.tf — étape 1 : l'image cible et le réseau

# L'image construite à partir de ../image-cible/Dockerfile
```

```
resource "docker_image" "cible" {
  name = "${var.prefixe}-cible:1.0"

  build {
    context = "${path.module}/../image-cible"
  }

  # Reconstruire l'image si le Dockerfile ou la clé autorisée changent
  triggers = {
    dockerfile = filesha1("${path.module}/../image-cible/Dockerfile")
    cle        = filesha1("${path.module}/../image-cible/authorized_keys")
  }

  # terraform destroy supprime aussi l'image
  keep_locally = false
}

# Le réseau privé des conteneurs : ils s'y joignent par leur NOM (DNS Docker)
resource "docker_network" "reseau" {
  name = "${var.prefixe}-net"
}
EOF
```

2.3 Initialiser, formater, valider

```
cd ~/tp-iac/terraform
terraform init
if terraform fmt -recursive -check; then echo "OK : code correctement formaté"; else echo "ERREUR : fichier(s) mal formaté(s) ci-d
        essus : recollez leur bloc"; fi
terraform validate
```

```
Initializing the backend...
Initializing provider plugins...
- Finding kreuzwerker/docker versions matching "4.6.0"...
- Finding hashicorp/local versions matching "2.9.1"...
- Installing kreuzwerker/docker v4.6.0...
- Installed kreuzwerker/docker v4.6.0 (self-signed, key ID 0DCE698927DAF8EC)
- Installing hashicorp/local v2.9.1...
- Installed hashicorp/local v2.9.1 (signed by HashiCorp)
...
Terraform has been successfully initialized!
...
OK : code correctement formaté
Success! The configuration is valid.
```

2.4 Planifier, lire, appliquer

On applique **toujours le plan qu'on a lu** : `terraform plan -out=tfplan` enregistre le plan, `terraform apply tfplan` applique exactement celui-là, sans question et sans surprise.

```
cd ~/tp-iac/terraform
terraform plan -out=tfplan
```

```
Terraform will perform the following actions:

  # docker_image.cible will be created
  + resource "docker_image" "cible" {
      + name         = "tp-iac-cible:1.0"
      ...
    }

  # docker_network.reseau will be created
  + resource "docker_network" "reseau" {
      + name        = "tp-iac-net"
      ...
    }

Plan: 2 to add, 0 to change, 0 to destroy.
```

Le plan annonce **2 créations** : l'image et le réseau. La construction de l'image prend 1 à 3 minutes (téléchargement d'Ubuntu et des paquets).

```
cd ~/tp-iac/terraform
terraform apply tfplan
```

```
docker_network.reseau: Creating...
docker_image.cible: Creating...
docker_network.reseau: Creation complete after 1s [id=…]
docker_image.cible: Still creating... [10s elapsed]
...
docker_image.cible: Creation complete after 35s [id=sha256:…tp-iac-cible:1.0]

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.
```

✅ Point de contrôle 2 :

```
cd ~/tp-iac/terraform
if docker image inspect tp-iac-cible:1.0 > /dev/null 2>&1; then echo "OK : l'image tp-iac-cible:1.0 existe"; else echo "ERREUR :
        l'image tp-iac-cible:1.0 est absente"; fi
if docker network inspect tp-iac-net > /dev/null 2>&1; then echo "OK : le réseau tp-iac-net existe"; else echo "ERREUR : le réseau
        tp-iac-net est absent"; fi
terraform state list
```

```
OK : l'image tp-iac-cible:1.0 existe
OK : le réseau tp-iac-net existe
docker_image.cible
docker_network.reseau
```

> 💬 **Question** : relancez `terraform plan`. Pourquoi annonce-t-il « No changes » alors que vous venez de créer deux objets ?

Partie 3 — Terraform : un module « serveur » et le parc (1 h)

3.1 Le module « serveur »

Un **module** est un dossier de fichiers `.tf` qu'on appelle comme une fonction : des **entrées** (`variables.tf`), un **traitement** (`main.tf`), des **sorties** (`outputs.tf`). Notre module crée **un** serveur ; on l'appellera autant de fois qu'il faut de serveurs.

`versions.tf` : un module **doit** déclarer les providers qu'il utilise. Sans ce fichier, Terraform chercherait un provider `hashicorp/docker` qui n'existe pas, et `terraform init` échouerait.

`~/tp-iac/terraform/modules/serveur/versions.tf`

```
cd ~/tp-iac
cat > terraform/modules/serveur/versions.tf <<'EOF'
# versions.tf — un module DOIT déclarer les providers qu'il utilise.
# Sans ce bloc, Terraform chercherait « hashicorp/docker », qui n'existe pas.
terraform {
  required_providers {
    docker = {
      source = "kreuzwerker/docker"
    }
  }
}
EOF
```

`variables.tf` : le **contrat** du module. `port_http` a une valeur par défaut `null` : il est donc facultatif.

`~/tp-iac/terraform/modules/serveur/variables.tf`

```
cd ~/tp-iac
cat > terraform/modules/serveur/variables.tf <<'EOF'
# variables.tf — les ENTRÉES du module « serveur » (son contrat)
variable "nom" {
  type        = string
  description = "Nom du conteneur (et nom DNS sur le réseau Docker)"
}

variable "image_id" {
  type        = string
  description = "Identifiant de l'image Docker à utiliser"
}

variable "reseau" {
  type        = string
  description = "Nom du réseau Docker à rejoindre"
}

variable "port_ssh" {
  type        = number
  description = "Port du poste redirigé vers le port 22 du conteneur"
}

variable "port_http" {
```

```
  type        = number
  description = "Port du poste redirigé vers le port 80 du conteneur (null = aucun)"
  default     = null
}
EOF
```

`main.tf` : le conteneur. Les points à comprendre :

- `privileged` et `tmpfs` : les réglages qui permettent à systemd de tourner dans un conteneur (repris de l'exercice du cours) ;
- `networks_advanced` : le conteneur rejoint le réseau `tp-iac-net` et y prend son **nom** comme nom DNS ;
- le premier bloc `ports` publie le SSH **uniquement sur `127.0.0.1`** : on ne peut s'y connecter que depuis votre poste ;
- le bloc **`dynamic "ports"`** fabrique **zéro ou un** bloc `ports` : si `port_http` vaut `null`, la liste `[]` est vide et aucun bloc n'est créé ; sinon la liste a un élément et un bloc est créé avec `ports.value`. C'est l'équivalent d'un `for_each` pour un **bloc** plutôt que pour une ressource.

 `~/tp-iac/terraform/modules/serveur/main.tf`

```
cd ~/tp-iac
cat > terraform/modules/serveur/main.tf <<'EOF'
# main.tf — un « serveur » = un conteneur systemd joignable en SSH
resource "docker_container" "serveur" {
  name     = var.nom
  hostname = var.nom
  image    = var.image_id

  # systemd a besoin de ces réglages pour tourner dans un conteneur
  privileged = true
  tmpfs = {
    "/run"      = ""
    "/run/lock" = ""
  }

  # Redémarre avec Docker (par exemple après un redémarrage de WSL)
  restart = "unless-stopped"

  networks_advanced {
    name = var.reseau
  }

  # SSH : uniquement depuis le poste (127.0.0.1), jamais depuis l'extérieur
  ports {
    internal = 22
    external = var.port_ssh
    ip       = "127.0.0.1"
  }

  # HTTP : bloc « dynamic » = 0 ou 1 bloc ports selon que port_http est fourni
  dynamic "ports" {
    for_each = var.port_http == null ? [] : [var.port_http]
    content {
      internal = 80
      external = ports.value
    }
  }
}
EOF
```

`outputs.tf` : ce que le module rend à celui qui l'appelle.

`~/tp-iac/terraform/modules/serveur/outputs.tf`

```
cd ~/tp-iac
cat > terraform/modules/serveur/outputs.tf <<'EOF'
# outputs.tf — les SORTIES du module « serveur »
output "nom" {
  description = "Nom du conteneur créé"
  value       = docker_container.serveur.name
}

output "port_ssh" {
  description = "Port SSH du poste vers ce conteneur"
  value       = var.port_ssh
}
EOF
```

3.2 Le parc de serveurs

La variable `serveurs` est une **map d'objets** : la clé est le nom court du serveur (`db1` , `api1` ...), la valeur décrit son groupe Ansible et ses ports. `optional(number)` rend `port_http` facultatif : seul `lb1` en a un.

`~/tp-iac/terraform/variables.tf`

```
cd ~/tp-iac
cat > terraform/variables.tf <<'EOF'
# variables.tf — les paramètres du projet
variable "prefixe" {
  type        = string
  description = "Préfixe de tous les noms Docker créés par ce projet"
  default     = "tp-iac"
}

variable "cle_ssh_privee" {
  type        = string
  description = "Clé privée qu'Ansible utilisera pour se connecter aux serveurs"
  default     = "~/.ssh/tp_iac_ed25519"
}

# Le parc de serveurs : une entrée = un conteneur.
#   groupe    : groupe Ansible du serveur (db, api ou lb)
#   port_ssh  : port du poste redirigé vers son SSH
#   port_http : port du poste redirigé vers son port 80 (facultatif)
variable "serveurs" {
  type = map(object({
    groupe    = string
    port_ssh  = number
    port_http = optional(number)
  }))
  description = "Serveurs à créer, indexés par leur nom court"

  default = {
    db1  = { groupe = "db", port_ssh = 2241 }
    api1 = { groupe = "api", port_ssh = 2242 }
    api2 = { groupe = "api", port_ssh = 2243 }
    lb1  = { groupe = "lb", port_ssh = 2244, port_http = 8080 }
```

```
  }
}
EOF
```

Dans `main.tf`, on ajoute l'appel du module avec **`for_each = var.serveurs`** : Terraform appelle le module **une fois par entrée** de la map, et `each.key` / `each.value` désignent l'entrée en cours. Les adresses dans le state seront `module.serveur["api1"]`, `module.serveur["db1"]` … : stables, contrairement à `count`.

 `~/tp-iac/terraform/main.tf`

```
cd ~/tp-iac
cat > terraform/main.tf <<'EOF'
# main.tf — étape 2 : image, réseau et parc de serveurs

# L'image construite à partir de ../image-cible/Dockerfile
resource "docker_image" "cible" {
  name = "${var.prefixe}-cible:1.0"

  build {
    context = "${path.module}/../image-cible"
  }

  # Reconstruire l'image si le Dockerfile ou la clé autorisée changent
  triggers = {
    dockerfile = filesha1("${path.module}/../image-cible/Dockerfile")
    cle        = filesha1("${path.module}/../image-cible/authorized_keys")
  }

  # terraform destroy supprime aussi l'image
  keep_locally = false
}

# Le réseau privé des conteneurs : ils s'y joignent par leur NOM (DNS Docker)
resource "docker_network" "reseau" {
  name = "${var.prefixe}-net"
}

# Un appel du module « serveur » PAR entrée de var.serveurs
module "serveur" {
  source   = "./modules/serveur"
  for_each = var.serveurs

  nom       = "${var.prefixe}-${each.key}"
  image_id  = docker_image.cible.image_id
  reseau    = docker_network.reseau.name
  port_ssh  = each.value.port_ssh
  port_http = each.value.port_http
}
EOF
```

`outputs.tf` : une expression `for` construit un résumé lisible de chaque serveur à partir des sorties du module.

 `~/tp-iac/terraform/outputs.tf`

```
cd ~/tp-iac
cat > terraform/outputs.tf <<'EOF'
# outputs.tf — ce que le projet affiche après un apply
```

```
output "conteneurs" {
  description = "Nom court => nom du conteneur et port SSH"
  value = {
    for nom, serveur in module.serveur :
    nom => "${serveur.nom} (ssh 127.0.0.1:${serveur.port_ssh})"
  }
}
EOF
```

3.3 Appliquer

Un **nouveau module** impose de relancer `terraform init` : Terraform doit l'enregistrer avant de pouvoir l'utiliser.

```
cd ~/tp-iac/terraform
terraform init
if terraform fmt -recursive -check; then echo "OK : code correctement formaté"; else echo "ERREUR : fichier(s) mal formaté(s) ci-d
        essus : recollez leur bloc"; fi
terraform validate
terraform plan -out=tfplan
```

```
  # module.serveur["api1"].docker_container.serveur will be created
  ...
  # module.serveur["api2"].docker_container.serveur will be created
  ...
  # module.serveur["db1"].docker_container.serveur will be created
  ...
  # module.serveur["lb1"].docker_container.serveur will be created
  ...
Plan: 4 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  + conteneurs = {
      + api1 = "tp-iac-api1 (ssh 127.0.0.1:2242)"
      + api2 = "tp-iac-api2 (ssh 127.0.0.1:2243)"
      + db1  = "tp-iac-db1 (ssh 127.0.0.1:2241)"
      + lb1  = "tp-iac-lb1 (ssh 127.0.0.1:2244)"
    }
```

Lisez le plan : **4 créations**, et **rien** sur l'image ni le réseau, qui existent déjà et n'ont pas changé.

```
cd ~/tp-iac/terraform
terraform apply tfplan
```

```
Apply complete! Resources: 4 added, 0 changed, 0 destroyed.

Outputs:

conteneurs = {
  "api1" = "tp-iac-api1 (ssh 127.0.0.1:2242)"
  "api2" = "tp-iac-api2 (ssh 127.0.0.1:2243)"
  "db1" = "tp-iac-db1 (ssh 127.0.0.1:2241)"
  "lb1" = "tp-iac-lb1 (ssh 127.0.0.1:2244)"
}
```

✅ **Point de contrôle 3** : les 4 conteneurs tournent, et votre clé ouvre chacun d'eux en SSH. La boucle laisse quelques secondes à SSH pour démarrer.

```
cd ~/tp-iac/terraform
docker ps --filter "name=tp-iac-" --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
sleep 5
for port in 2241 2242 2243 2244; do
  nom=$(ssh -i ~/.ssh/tp_iac_ed25519 -p "$port" -o StrictHostKeyChecking=no -o UserKnownHostsFile=/dev/null -o LogLevel=ERROR -o C
        onnectTimeout=5 ansible@127.0.0.1 hostname 2> /dev/null)
  if [ -n "$nom" ]; then echo "OK : port $port -> connexion SSH à $nom"; else echo "ERREUR : connexion SSH impossible sur le port
        $port (fiche D6)"; fi
done
```

```
NAMES         STATUS          PORTS
tp-iac-api1   Up 20 seconds   127.0.0.1:2242->22/tcp
tp-iac-api2   Up 20 seconds   127.0.0.1:2243->22/tcp
tp-iac-db1    Up 20 seconds   127.0.0.1:2241->22/tcp
tp-iac-lb1    Up 20 seconds   127.0.0.1:2244->22/tcp, 0.0.0.0:8080->80/tcp
OK : port 2241 -> connexion SSH à tp-iac-db1
OK : port 2242 -> connexion SSH à tp-iac-api1
OK : port 2243 -> connexion SSH à tp-iac-api2
OK : port 2244 -> connexion SSH à tp-iac-lb1
```

💬 **Question** : seul `tp-iac-lb1` publie le port `8080`. Quelle ligne de quel fichier en décide ?

Partie 4 — Terraform écrit l'inventaire Ansible (45 min)

Dans l'exercice Ansible du cours, l'inventaire `inventory.ini` était écrit **à la main** : chaque port recopié depuis le code Terraform. Ajoutez un serveur et il faut penser à modifier deux fichiers. Ici, **Terraform écrit l'inventaire lui-même**, à partir de la même variable `serveurs` : une seule source de vérité.

4.1 Le gabarit d'inventaire

Un fichier `.tftpl` est un **gabarit** : du texte avec des trous que `templatefile()` remplit.

- `${…}` insère une valeur ;
- `%{ for … }` … `%{ endfor }` répète un morceau de texte ;
- le `~` colle la ligne suivante, pour ne pas laisser de lignes vides parasites.

Pour chaque serveur, on écrit son port SSH **et** une variable `nom_conteneur` : c'est par ce nom que les serveurs se joignent sur le réseau Docker. Ansible s'en servira pour dire aux API où est la base, et à nginx où sont les API.

📄 `~/tp-iac/terraform/templates/inventory.ini.tftpl`

```
cd ~/tp-iac
cat > terraform/templates/inventory.ini.tftpl <<'EOF'
# Fichier GÉNÉRÉ par Terraform (terraform/templates/inventory.ini.tftpl).
# Ne le modifiez pas à la main : il est réécrit à chaque terraform apply.
%{ for groupe, noms in groupes ~}

[${groupe}]
%{ for nom in noms ~}
${nom} ansible_host=127.0.0.1 ansible_port=${serveurs[nom].port_ssh} nom_conteneur=${prefixe}-${nom}
%{ endfor ~}
%{ endfor ~}

[all:vars]
ansible_user=ansible
ansible_ssh_private_key_file=${cle_ssh}
EOF
```

4.2 Générer le fichier

Dans `main.tf`, deux ajouts à la fin :

- un bloc **`locals`** qui regroupe les noms de serveurs par groupe Ansible, avec des expressions `for` ; `distinct()` retire les doublons et `sort()` trie. Résultat : `{ api = ["api1", "api2"], db = ["db1"], lb = ["lb1"] }` ;
- une ressource **`local_file`** (provider `local`) qui écrit le résultat de `templatefile()` dans `../ansible/inventory.ini`. Le `depends_on` garantit que l'inventaire n'est écrit qu'**après** la création des conteneurs.

`~/tp-iac/terraform/main.tf`

```
cd ~/tp-iac
cat > terraform/main.tf <<'EOF'
# main.tf — image, réseau, parc de serveurs et inventaire Ansible

# L'image construite à partir de ../image-cible/Dockerfile
resource "docker_image" "cible" {
  name = "${var.prefixe}-cible:1.0"

  build {
    context = "${path.module}/../image-cible"
  }

  # Reconstruire l'image si le Dockerfile ou la clé autorisée changent
  triggers = {
    dockerfile = filesha1("${path.module}/../image-cible/Dockerfile")
    cle        = filesha1("${path.module}/../image-cible/authorized_keys")
  }

  # terraform destroy supprime aussi l'image
  keep_locally = false
}

# Le réseau privé des conteneurs : ils s'y joignent par leur NOM (DNS Docker)
resource "docker_network" "reseau" {
  name = "${var.prefixe}-net"
```

```
}

# Un appel du module « serveur » PAR entrée de var.serveurs
module "serveur" {
  source   = "./modules/serveur"
  for_each = var.serveurs

  nom       = "${var.prefixe}-${each.key}"
  image_id  = docker_image.cible.image_id
  reseau    = docker_network.reseau.name
  port_ssh  = each.value.port_ssh
  port_http = each.value.port_http
}

# Regrouper les noms de serveurs par groupe Ansible :
#   { api = ["api1", "api2"], db = ["db1"], lb = ["lb1"] }
locals {
  groupes = {
    for groupe in distinct([for s in values(var.serveurs) : s.groupe]) :
    groupe => sort([for nom, s in var.serveurs : nom if s.groupe == groupe])
  }
}

# L'inventaire Ansible, ÉCRIT par Terraform à partir d'un gabarit
resource "local_file" "inventaire" {
  filename        = "${path.module}/../ansible/inventory.ini"
  file_permission = "0644"

  content = templatefile("${path.module}/templates/inventory.ini.tftpl", {
    groupes  = local.groupes
    serveurs = var.serveurs
    prefixe  = var.prefixe
    cle_ssh  = var.cle_ssh_privee
  })

  # N'écrire l'inventaire qu'une fois les conteneurs créés
  depends_on = [module.serveur]
}
EOF
```

`outputs.tf` gagne deux sorties : le chemin de l'inventaire et l'URL de StockLine.

`~/tp-iac/terraform/outputs.tf`

```
cd ~/tp-iac
cat > terraform/outputs.tf <<'EOF'
# outputs.tf — ce que le projet affiche après un apply
output "conteneurs" {
  description = "Nom court => nom du conteneur et port SSH"
  value = {
    for nom, serveur in module.serveur :
    nom => "${serveur.nom} (ssh 127.0.0.1:${serveur.port_ssh})"
  }
}

output "inventaire_ansible" {
  description = "Chemin de l'inventaire généré pour Ansible"
  value       = abspath(local_file.inventaire.filename)
}

output "url_stockline" {
  description = "URL publique de StockLine (via le répartiteur nginx)"
  value       = "http://localhost:8080"
```

```
}
EOF
```

```
cd ~/tp-iac/terraform
if terraform fmt -recursive -check; then echo "OK : code correctement formaté"; else echo "ERREUR : fichier(s) mal formaté(s) ci-d
        essus : recollez leur bloc"; fi
terraform validate
terraform plan -out=tfplan
```

```
  # local_file.inventaire will be created
  + resource "local_file" "inventaire" {
      + content              = <<-EOT
            # Fichier GÉNÉRÉ par Terraform (terraform/templates/inventory.ini.tftpl).
            ...
        EOT
      + filename             = "./../ansible/inventory.ini"
      ...
    }

Plan: 1 to add, 0 to change, 0 to destroy.
```

```
cd ~/tp-iac/terraform
terraform apply tfplan
```

✅ Point de contrôle 4 :

```
cd ~/tp-iac/terraform
terraform output
cat ~/tp-iac/ansible/inventory.ini
```

```
conteneurs = {
  "api1" = "tp-iac-api1 (ssh 127.0.0.1:2242)"
  "api2" = "tp-iac-api2 (ssh 127.0.0.1:2243)"
  "db1" = "tp-iac-db1 (ssh 127.0.0.1:2241)"
  "lb1" = "tp-iac-lb1 (ssh 127.0.0.1:2244)"
}
inventaire_ansible = "/home/vous/tp-iac/ansible/inventory.ini"
url_stockline = "http://localhost:8080"
# Fichier GÉNÉRÉ par Terraform (terraform/templates/inventory.ini.tftpl).
# Ne le modifiez pas à la main : il est réécrit à chaque terraform apply.

[api]
api1 ansible_host=127.0.0.1 ansible_port=2242 nom_conteneur=tp-iac-api1
api2 ansible_host=127.0.0.1 ansible_port=2243 nom_conteneur=tp-iac-api2

[db]
db1 ansible_host=127.0.0.1 ansible_port=2241 nom_conteneur=tp-iac-db1

[lb]
lb1 ansible_host=127.0.0.1 ansible_port=2244 nom_conteneur=tp-iac-lb1

[all:vars]
```

```
ansible_user=ansible
ansible_ssh_private_key_file=~/.ssh/tp_iac_ed25519
```

> 💬 **Question** : ouvrez `inventory.ini`, modifiez un port à la main, puis relancez `terraform plan`. Que propose Terraform ? Qu'en concluez-vous sur le rôle de ce fichier ? (Faites `terraform apply` pour le remettre en ordre avant de continuer.)

Partie 5 — Ansible : configuration du projet et rôle « commun » (45 min)

5.1 La configuration du projet

`ansible.cfg` est lu par Ansible **parce que vous lancez vos commandes depuis le dossier `ansible`**. Il désigne l'inventaire généré, le dossier des rôles, et règle SSH. Les conteneurs étant recréés souvent avec de nouvelles clés d'hôte sur les mêmes ports, on ne mémorise pas ces clés : c'est acceptable ici parce que tout est sur `127.0.0.1`, **jamais** face à de vrais serveurs.

📄 `~/tp-iac/ansible/ansible.cfg`

```
cd ~/tp-iac
cat > ansible/ansible.cfg <<'EOF'
# ansible.cfg — configuration du projet.
# Ansible la lit parce que vous lancez les commandes DEPUIS ce dossier.
[defaults]
# L'inventaire généré par Terraform
inventory = inventory.ini
# Où chercher les rôles du projet
roles_path = roles
# Ne pas afficher d'avertissement sur le choix de l'interpréteur Python
interpreter_python = auto_silent

[ssh_connection]
# Les conteneurs sont recréés souvent, avec de NOUVELLES clés d'hôte sur les
# MÊMES ports : on ne mémorise pas ces clés (acceptable ici car tout est local,
# sur 127.0.0.1 ; ne JAMAIS faire cela face à de vrais serveurs).
ssh_args = -o ControlMaster=auto -o ControlPersist=60s -o UserKnownHostsFile=/dev/null -o StrictHostKeyChecking=no
# Moins d'allers-retours SSH : exécution nettement plus rapide
pipelining = True
EOF
```

`group_vars/all/main.yml` : les variables communes à **tous** les serveurs (groupe `all`). Pour l'instant, une seule.

📄 `~/tp-iac/ansible/group_vars/all/main.yml`

```
cd ~/tp-iac
cat > ansible/group_vars/all/main.yml <<'EOF'
# group_vars/all/main.yml — variables communes à TOUS les serveurs
---
environnement: formation
EOF
```

Premier contact : Ansible voit-il les 4 serveurs, et peut-il s'y connecter ?

```
cd ~/tp-iac/ansible
ansible-inventory --graph
ansible all -m ansible.builtin.ping
```

```
@all:
  |--@ungrouped:
  |--@api:
  |  |--api1
  |  |--api2
  |--@db:
  |  |--db1
  |--@lb:
  |  |--lb1
db1 | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3.12"
    },
    "changed": false,
    "ping": "pong"
}
...
```

Les 4 serveurs répondent `"ping": "pong"` (l'ordre peut varier). La ligne `discovered_interpreter_python` indique le Python qu'Ansible a trouvé sur la cible.

5.2 Le rôle « commun »

Un **rôle** est un dossier à structure imposée. Ansible y cherche lui-même les fichiers :

Dossier	Contenu
`defaults/main.yml`	les valeurs par défaut des variables du rôle (précédence la plus faible : tout peut les écraser)
`tasks/main.yml`	les tâches
`templates/`	les gabarits Jinja2 utilisés par le module `template`
`handlers/main.yml`	les handlers (parties 6 à 8)

Le rôle `commun` installe quelques paquets sur **tous** les serveurs et dépose un message d'accueil (`/etc/motd`) généré par un **template**. À la différence de `copy` , le module `template` remplace les `{{ … }}` par des valeurs **propres à chaque serveur** : son nom d'inventaire, ses groupes, ses *facts*. Les *facts* (`ansible_facts`) sont les informations qu'Ansible collecte automatiquement sur chaque machine en début de play.

```
cd ~/tp-iac/ansible
mkdir -p roles/commun/defaults roles/commun/tasks roles/commun/templates
```

 `~/tp-iac/ansible/roles/commun/defaults/main.yml`

```
cd ~/tp-iac
cat > ansible/roles/commun/defaults/main.yml <<'EOF'
# roles/commun/defaults/main.yml — valeurs par défaut du rôle (précédence la plus faible)
---
commun_paquets:
  - curl
  - ca-certificates
  - acl
  - less
  - iproute2
EOF
```

 `~/tp-iac/ansible/roles/commun/tasks/main.yml`

```
cd ~/tp-iac
cat > ansible/roles/commun/tasks/main.yml <<'EOF'
# roles/commun/tasks/main.yml — le socle installé sur TOUS les serveurs
---
- name: Mettre à jour le cache apt (au plus une fois par heure)
  ansible.builtin.apt:
    update_cache: true
    cache_valid_time: 3600

- name: Installer les paquets communs
  ansible.builtin.apt:
    name: "{{ commun_paquets }}"
    state: present

- name: Déposer le message d'accueil (motd)
  ansible.builtin.template:
    src: motd.j2
    dest: /etc/motd
    owner: root
    group: root
    mode: "0644"
EOF
```

 `~/tp-iac/ansible/roles/commun/templates/motd.j2`

```
cd ~/tp-iac
cat > ansible/roles/commun/templates/motd.j2 <<'EOF'
# {{ ansible_managed }}
============================================================
 Serveur       : {{ inventory_hostname }} ({{ nom_conteneur }})
 Groupe(s)     : {{ group_names | join(', ') }}
 Environnement : {{ environnement }}
 Système       : {{ ansible_facts['distribution'] }} {{ ansible_facts['distribution_version'] }}
 Configuré par Ansible : toute modification manuelle sera écrasée.
============================================================
EOF
```

5.3 Le playbook du projet

`site.yml` est **le** playbook du projet. Chaque play associe un **groupe** de l'inventaire à des **rôles**. Il grandira à chaque partie.

 `~/tp-iac/ansible/site.yml`

```
cd ~/tp-iac
cat > ansible/site.yml <<'EOF'
# site.yml — LE playbook du projet : chaque play applique des rôles à un groupe
---
- name: Socle commun à tous les serveurs
  hosts: all
  become: true
  roles:
    - commun
EOF
```

```
cd ~/tp-iac/ansible
ansible-playbook site.yml
```

```
PLAY [Socle commun à tous les serveurs] ************************************

TASK [Gathering Facts] *****************************************************
ok: [db1]
...
TASK [commun : Installer les paquets communs] ******************************
changed: [db1]
...
PLAY RECAP *****************************************************************
api1                       : ok=4    changed=2    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
api2                       : ok=4    changed=2    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
db1                        : ok=4    changed=2    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
lb1                        : ok=4    changed=2    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

5.4 L'idempotence

Un playbook **idempotent** décrit un **état** : le rejouer sur des serveurs déjà conformes ne change **rien** (`changed=0`). Vérifions-le ; on rejouera ce même contrôle à chaque partie.

```
cd ~/tp-iac/ansible
ansible-playbook site.yml | tee /tmp/tp-iac-rejeu.log
if grep -Eq "changed=[1-9]|failed=[1-9]|unreachable=[1-9]" /tmp/tp-iac-rejeu.log; then echo "ERREUR : le rejeu a encore changé que
        lque chose (relisez le PLAY RECAP)"; else echo "OK : rejeu sans aucun changement, le playbook est idempotent"; fi
```

✅ **Point de contrôle 5** : la dernière ligne est `OK : rejeu sans aucun changement…` , puis le message d'accueil est bien personnalisé par serveur :

```
cd ~/tp-iac/ansible
ansible api1 -m ansible.builtin.command -a "cat /etc/motd"
```

```
api1 | CHANGED | rc=0 >>
# Ansible managed
============================================================
 Serveur       : api1 (tp-iac-api1)
 Groupe(s)     : api
 Environnement : formation
 Système       : Ubuntu 24.04
 Configuré par Ansible : toute modification manuelle sera écrasée.
============================================================
```

> 💬 **Question** : le module `command` affiche `CHANGED` alors qu'un simple `cat` ne modifie rien. Pourquoi ? Que faudrait-il ajouter dans un playbook pour que ce soit juste ?

Partie 6 — Coffre chiffré et rôle « postgresql » (45 min)

6.1 Le coffre (ansible-vault)

La base aura un mot de passe. Il ne doit **jamais** apparaître en clair dans le projet. **ansible-vault** chiffre un fichier de variables avec un mot de passe « maître ». On range ce mot de passe maître dans `.vault-pass` (déjà listé dans `.gitignore` , droits `600`) et `ansible.cfg` indique à Ansible où le trouver.

D'abord, `ansible.cfg` gagne la ligne `vault_password_file` :

📄 `~/tp-iac/ansible/ansible.cfg`

```
cd ~/tp-iac
cat > ansible/ansible.cfg <<'EOF'
# ansible.cfg — configuration du projet.
# Ansible la lit parce que vous lancez les commandes DEPUIS ce dossier.
[defaults]
# L'inventaire généré par Terraform
inventory = inventory.ini
# Où chercher les rôles du projet
roles_path = roles
# Ne pas afficher d'avertissement sur le choix de l'interpréteur Python
interpreter_python = auto_silent
# Fichier contenant le mot de passe du coffre (JAMAIS commité)
vault_password_file = .vault-pass

[ssh_connection]
# Les conteneurs sont recréés souvent, avec de NOUVELLES clés d'hôte sur les
# MÊMES ports : on ne mémorise pas ces clés (acceptable ici car tout est local,
# sur 127.0.0.1 ; ne JAMAIS faire cela face à de vrais serveurs).
ssh_args = -o ControlMaster=auto -o ControlPersist=60s -o UserKnownHostsFile=/dev/null -o StrictHostKeyChecking=no
# Moins d'allers-retours SSH : exécution nettement plus rapide
pipelining = True
EOF
```

Ensuite on génère **deux** secrets aléatoires avec `openssl rand` : le mot de passe maître du coffre, et le mot de passe de la base, qu'on écrit dans `vault.yml` puis qu'on chiffre aussitôt. On le génère en hexadécimal (chiffres et lettres a à f) pour qu'il ne contienne aucun caractère spécial gênant dans une URL de connexion.

```
cd ~/tp-iac/ansible
test -f .vault-pass || openssl rand -base64 32 > .vault-pass
chmod 600 .vault-pass
printf -- '---\nvault_stockline_db_mot_de_passe: "%s"\n' "$(openssl rand -hex 16)" > group_vars/all/vault.yml
ansible-vault encrypt group_vars/all/vault.yml
head -n 1 group_vars/all/vault.yml
ansible-vault view group_vars/all/vault.yml
```

```
Encryption successful
$ANSIBLE_VAULT;1.1;AES256
---
vault_stockline_db_mot_de_passe: "3f9c0a…"
```

Sur le disque, le fichier est chiffré (`$ANSIBLE_VAULT;1.1;AES256`) ; `ansible-vault view` le déchiffre à l'écran grâce à `.vault-pass` . Pour le modifier plus tard : `ansible-vault edit group_vars/all/vault.yml` .

6.2 Les variables de la base

`group_vars/all/main.yml` est complété. Observez le **motif d'indirection** : `stockline_db_mot_de_passe` ne contient pas le secret, il **pointe** vers `vault_stockline_db_mot_de_passe` (dans le coffre). Un `grep` dans les fichiers en clair montre **où** le secret est utilisé, sans jamais le révéler.

`stockline_db_hote` va chercher dans l'inventaire la variable `nom_conteneur` du premier serveur du groupe `db` : l'adresse de la base n'est écrite nulle part en dur.

`~/tp-iac/ansible/group_vars/all/main.yml`

```
cd ~/tp-iac
cat > ansible/group_vars/all/main.yml <<'EOF'
# group_vars/all/main.yml — variables communes à TOUS les serveurs
---
environnement: formation

# Base de données de StockLine
stockline_db_nom: stockline
stockline_db_utilisateur: stockline

# Le mot de passe vit CHIFFRÉ dans vault.yml : ici, on ne fait que le référencer
stockline_db_mot_de_passe: "{{ vault_stockline_db_mot_de_passe }}"

# Le serveur de base de données = le premier membre du groupe « db »,
# joint par son nom de conteneur sur le réseau Docker
stockline_db_hote: "{{ hostvars[groups['db'][0]]['nom_conteneur'] }}"

# Port d'écoute de l'API sur chaque serveur du groupe « api »
stockline_port: 8000
EOF
```

6.3 Le rôle « postgresql »

Ce rôle introduit les **handlers**. Modifier la configuration de PostgreSQL ne sert à rien tant que le service n'a pas redémarré. Mais redémarrer à **chaque** exécution serait brutal. Une tâche qui modifie un fichier **notifie** (`notify`) le handler « Redémarrer PostgreSQL ». Le handler ne s'exécute **que si** une tâche notifiante a effectivement changé quelque chose, **une seule fois**, à la fin du play.

Les points clés des tâches :

- `listen_addresses = '*'` : par défaut PostgreSQL n'écoute que sur `localhost` ; les API sont sur d'autres conteneurs ;
- la ligne `pg_hba.conf` autorise l'utilisateur `stockline` sur la base `stockline` depuis `samenet` (tout le sous-réseau du serveur, c'est-à-dire le réseau Docker) avec un mot de passe chiffré `scram-sha-256` ;
- `postgresql_user` et `postgresql_db` viennent de la collection `community.postgresql` . Ils s'exécutent en tant qu'utilisateur système `postgres` (`become_user: postgres`). `no_log: true` empêche le mot de passe d'apparaître dans la sortie.

```
cd ~/tp-iac/ansible
mkdir -p roles/postgresql/defaults roles/postgresql/tasks roles/postgresql/handlers
```

`~/tp-iac/ansible/roles/postgresql/defaults/main.yml`

```
cd ~/tp-iac
cat > ansible/roles/postgresql/defaults/main.yml <<'EOF'
# roles/postgresql/defaults/main.yml
---
```

```
# Ubuntu 24.04 fournit PostgreSQL 16
postgresql_version: "16"
postgresql_repertoire_conf: "/etc/postgresql/{{ postgresql_version }}/main"
EOF
```

~/tp-iac/ansible/roles/postgresql/tasks/main.yml

```
cd ~/tp-iac
cat > ansible/roles/postgresql/tasks/main.yml <<'EOF'
# roles/postgresql/tasks/main.yml — le serveur de base de données
---
- name: Installer PostgreSQL et le pilote Python dont Ansible a besoin
  ansible.builtin.apt:
    name:
      - postgresql
      - python3-psycopg2
    state: present

- name: Démarrer PostgreSQL et l'activer au démarrage
  ansible.builtin.systemd_service:
    name: postgresql
    state: started
    enabled: true

- name: Écouter sur toutes les interfaces (pas seulement localhost)
  ansible.builtin.lineinfile:
    path: "{{ postgresql_repertoire_conf }}/postgresql.conf"
    regexp: "^#?listen_addresses\\s*="
    line: "listen_addresses = '*'"
  notify: Redémarrer PostgreSQL

- name: Autoriser l'utilisateur StockLine depuis le réseau Docker (samenet)
  ansible.builtin.lineinfile:
    path: "{{ postgresql_repertoire_conf }}/pg_hba.conf"
    line: "host    {{ stockline_db_nom }}    {{ stockline_db_utilisateur }}    samenet    scram-sha-256"
  notify: Redémarrer PostgreSQL

- name: Créer l'utilisateur de la base StockLine
  community.postgresql.postgresql_user:
    name: "{{ stockline_db_utilisateur }}"
    password: "{{ stockline_db_mot_de_passe }}"
  become: true
  become_user: postgres
  no_log: true

- name: Créer la base StockLine (propriété de l'utilisateur stockline)
  community.postgresql.postgresql_db:
    name: "{{ stockline_db_nom }}"
    owner: "{{ stockline_db_utilisateur }}"
  become: true
  become_user: postgres
EOF
```

~/tp-iac/ansible/roles/postgresql/handlers/main.yml

```
cd ~/tp-iac
cat > ansible/roles/postgresql/handlers/main.yml <<'EOF'
# roles/postgresql/handlers/main.yml
# Un handler ne s'exécute QUE s'il a été notifié par une tâche « changed »,
```

```
# une seule fois, à la fin du play.
---
- name: Redémarrer PostgreSQL
  ansible.builtin.systemd_service:
    name: postgresql
    state: restarted
EOF
```

Le playbook gagne un play pour le groupe `db` :

📄 `~/tp-iac/ansible/site.yml`

```
cd ~/tp-iac
cat > ansible/site.yml <<'EOF'
# site.yml — LE playbook du projet : chaque play applique des rôles à un groupe
---
- name: Socle commun à tous les serveurs
  hosts: all
  become: true
  roles:
    - commun

- name: Serveur de base de données
  hosts: db
  become: true
  roles:
    - postgresql
EOF
```

```
cd ~/tp-iac/ansible
ansible-playbook site.yml
```

```
...
TASK [postgresql : Écouter sur toutes les interfaces (pas seulement localhost)] ***
changed: [db1]
...
RUNNING HANDLER [postgresql : Redémarrer PostgreSQL] ***************************
changed: [db1]

PLAY RECAP *********************************************************************
api1                       : ok=4    changed=0    unreachable=0    failed=0 ...
api2                       : ok=4    changed=0    unreachable=0    failed=0 ...
db1                        : ok=12   changed=7    unreachable=0    failed=0 ...
lb1                        : ok=4    changed=0    unreachable=0    failed=0 ...
```

Remarquez : `api1`, `api2` et `lb1` sont à `changed=0`. Le rôle `commun` est déjà conforme chez eux, Ansible ne refait rien.

✅ **Point de contrôle 6** :

```
cd ~/tp-iac/ansible
ansible-playbook site.yml | tee /tmp/tp-iac-rejeu.log
```

```
if grep -Eq "changed=[1-9]|failed=[1-9]|unreachable=[1-9]" /tmp/tp-iac-rejeu.log; then echo "ERREUR : le rejeu a encore changé que
        lque chose (relisez le PLAY RECAP)"; else echo "OK : rejeu sans aucun changement, le playbook est idempotent"; fi
if ansible db1 -b -m ansible.builtin.command -a "ss -ltn" | grep -q "0.0.0.0:5432"; then echo "OK : PostgreSQL écoute sur le résea
        u (port 5432)"; else echo "ERREUR : PostgreSQL n'écoute pas sur le réseau"; fi
if ansible db1 -b --become-user postgres -m ansible.builtin.command -a "psql -tAc 'SELECT datname FROM pg_database'" | grep -qx "s
        tockline"; then echo "OK : la base stockline existe"; else echo "ERREUR : la base stockline est absente"; fi
```

```
...
OK : rejeu sans aucun changement, le playbook est idempotent
OK : PostgreSQL écoute sur le réseau (port 5432)
OK : la base stockline existe
```

> 💬 **Question** : au deuxième passage, le handler « Redémarrer PostgreSQL » ne s'est pas exécuté. Pourquoi ?

Partie 7 — Rôle « stockline » : l'API (45 min)

Ce rôle installe l'API comme un vrai service Linux, sur chaque serveur du groupe `api` :

1. un **utilisateur système** `stockline`, sans shell : l'API ne tourne jamais en `root` ;
2. le code copié dans `/opt/stockline/app` et ses dépendances installées dans un **environnement virtuel** Python (`venv`), plus le pilote PostgreSQL `psycopg2-binary`. Le paquet `python3-packaging` est installé d'abord : le module `pip` d'Ansible en a besoin **sur le serveur cible** ;
3. un **fichier d'environnement** `/etc/stockline/stockline.env` (droits `0640`, lisible par `root` et le groupe `stockline` uniquement) qui contient `DATABASE_URL`, donc le mot de passe. `diff: false` empêche son contenu de s'afficher avec `--diff` ;
4. une **unité systemd** générée par template : systemd démarre l'API, la redémarre si elle plante, et la relance au boot ;
5. une **vérification** finale : le module `uri` appelle `/sante` jusqu'à obtenir `"base_de_donnees": "ok"` (au plus 15 essais espacés de 2 s).

`meta: flush_handlers` exécute **tout de suite** les handlers en attente, ici le redémarrage, pour que la vérification teste bien la nouvelle version.

Dans `site.yml`, le play `api` porte `serial: 1` : Ansible configure les serveurs d'API **l'un après l'autre**. C'est un déploiement progressif : derrière un répartiteur, il reste toujours une API en service pendant une mise à jour.

```
cd ~/tp-iac/ansible
mkdir -p roles/stockline/defaults roles/stockline/tasks roles/stockline/handlers roles/stockline/templates
```

`~/tp-iac/ansible/roles/stockline/defaults/main.yml`

```
cd ~/tp-iac
cat > ansible/roles/stockline/defaults/main.yml <<'EOF'
# roles/stockline/defaults/main.yml
---
stockline_repertoire: /opt/stockline
stockline_utilisateur: stockline
EOF
```

 `~/tp-iac/ansible/roles/stockline/tasks/main.yml`

```
cd ~/tp-iac
cat > ansible/roles/stockline/tasks/main.yml <<'EOF'
# roles/stockline/tasks/main.yml — l'API StockLine, servie par systemd
---
- name: Installer venv (environnements virtuels) et packaging (requis par le module pip)
  ansible.builtin.apt:
    name:
      - python3-venv
      - python3-packaging
    state: present

- name: Créer l'utilisateur système qui fera tourner l'API
  ansible.builtin.user:
    name: "{{ stockline_utilisateur }}"
    system: true
    shell: /usr/sbin/nologin
    home: "{{ stockline_repertoire }}"
    create_home: false

- name: Créer le répertoire de l'application
  ansible.builtin.file:
    path: "{{ stockline_repertoire }}"
    state: directory
    owner: root
    group: root
    mode: "0755"

- name: Copier le code de l'application (dossier stockline/app du poste)
  ansible.builtin.copy:
    src: "{{ playbook_dir }}/../stockline/app"
    dest: "{{ stockline_repertoire }}/"
    owner: root
    group: root
    mode: "0644"
    directory_mode: "0755"
  notify: Redémarrer StockLine

- name: Copier la liste des dépendances Python
  ansible.builtin.copy:
    src: "{{ playbook_dir }}/../stockline/requirements.txt"
    dest: "{{ stockline_repertoire }}/requirements.txt"
    owner: root
    group: root
    mode: "0644"

- name: Installer les dépendances dans un environnement virtuel
  ansible.builtin.pip:
    requirements: "{{ stockline_repertoire }}/requirements.txt"
    virtualenv: "{{ stockline_repertoire }}/venv"
    virtualenv_command: python3 -m venv
```

```
  notify: Redémarrer StockLine

- name: Installer le pilote PostgreSQL dans le même environnement virtuel
  ansible.builtin.pip:
    name: psycopg2-binary==2.9.10
    virtualenv: "{{ stockline_repertoire }}/venv"
    virtualenv_command: python3 -m venv
  notify: Redémarrer StockLine

- name: Créer le répertoire de configuration (lisible par stockline seulement)
  ansible.builtin.file:
    path: /etc/stockline
    state: directory
    owner: root
    group: "{{ stockline_utilisateur }}"
    mode: "0750"

- name: Déposer le fichier d'environnement (il contient le mot de passe)
  ansible.builtin.template:
    src: stockline.env.j2
    dest: /etc/stockline/stockline.env
    owner: root
    group: "{{ stockline_utilisateur }}"
    mode: "0640"
  diff: false
  notify: Redémarrer StockLine

- name: Déposer l'unité systemd
  ansible.builtin.template:
    src: stockline.service.j2
    dest: /etc/systemd/system/stockline.service
    owner: root
    group: root
    mode: "0644"
  notify: Redémarrer StockLine

- name: Démarrer StockLine et l'activer au démarrage
  ansible.builtin.systemd_service:
    name: stockline
    state: started
    enabled: true
    daemon_reload: true

- name: Exécuter maintenant les redémarrages en attente
  ansible.builtin.meta: flush_handlers

- name: Vérifier que l'API répond ET joint la base de données
  ansible.builtin.uri:
    url: "http://127.0.0.1:{{ stockline_port }}/sante"
    return_content: true
  register: sante
  until: sante.json is defined and sante.json.base_de_donnees == "ok"
  retries: 15
  delay: 2
EOF
```

 `~/tp-iac/ansible/roles/stockline/handlers/main.yml`

```
cd ~/tp-iac
cat > ansible/roles/stockline/handlers/main.yml <<'EOF'
# roles/stockline/handlers/main.yml
---
- name: Redémarrer StockLine
```

```
  ansible.builtin.systemd_service:
    name: stockline
    state: restarted
    daemon_reload: true
EOF
```

`~/tp-iac/ansible/roles/stockline/templates/stockline.env.j2`

```
cd ~/tp-iac
cat > ansible/roles/stockline/templates/stockline.env.j2 <<'EOF'
# {{ ansible_managed }}
# Lu par systemd (EnvironmentFile) : variables d'environnement de l'API
DATABASE_URL=postgresql+psycopg2://{{ stockline_db_utilisateur }}:{{ stockline_db_mot_de_passe }}@{{ stockline_db_hote }}:5432/{{
        stockline_db_nom }}
EOF
```

`~/tp-iac/ansible/roles/stockline/templates/stockline.service.j2`

```
cd ~/tp-iac
cat > ansible/roles/stockline/templates/stockline.service.j2 <<'EOF'
# {{ ansible_managed }}
[Unit]
Description=StockLine - API d'inventaire
After=network-online.target
Wants=network-online.target

[Service]
User={{ stockline_utilisateur }}
Group={{ stockline_utilisateur }}
WorkingDirectory={{ stockline_repertoire }}
EnvironmentFile=/etc/stockline/stockline.env
ExecStart={{ stockline_repertoire }}/venv/bin/uvicorn app.main:app --host 0.0.0.0 --port {{ stockline_port }}
Restart=on-failure
RestartSec=2

[Install]
WantedBy=multi-user.target
EOF
```

`~/tp-iac/ansible/site.yml`

```
cd ~/tp-iac
cat > ansible/site.yml <<'EOF'
# site.yml — LE playbook du projet : chaque play applique des rôles à un groupe
---
- name: Socle commun à tous les serveurs
  hosts: all
  become: true
  roles:
    - commun

- name: Serveur de base de données
  hosts: db
  become: true
  roles:
```

```
    - postgresql

- name: Serveurs d'API StockLine (un par un)
  hosts: api
  become: true
  serial: 1
  roles:
    - stockline
EOF
```

L'installation des dépendances Python prend 1 à 3 minutes par serveur.

```
cd ~/tp-iac/ansible
ansible-playbook site.yml
```

```
...
PLAY [Serveurs d'API StockLine (un par un)] ************************************
...
TASK [stockline : Vérifier que l'API répond ET joint la base de données] *******
ok: [api1]
...
TASK [stockline : Vérifier que l'API répond ET joint la base de données] *******
ok: [api2]

PLAY RECAP *********************************************************************
api1                       : ok=18   changed=12   unreachable=0    failed=0 ...
api2                       : ok=18   changed=12   unreachable=0    failed=0 ...
db1                        : ok=11   changed=0    unreachable=0    failed=0 ...
lb1                        : ok=4    changed=0    unreachable=0    failed=0 ...
```

✅ Point de contrôle 7 :

```
cd ~/tp-iac/ansible
ansible-playbook site.yml | tee /tmp/tp-iac-rejeu.log
if grep -Eq "changed=[1-9]|failed=[1-9]|unreachable=[1-9]" /tmp/tp-iac-rejeu.log; then echo "ERREUR : le rejeu a encore changé que
        lque chose (relisez le PLAY RECAP)"; else echo "OK : rejeu sans aucun changement, le playbook est idempotent"; fi
for serveur in api1 api2; do
  if ansible "$serveur" -m ansible.builtin.command -a "curl -s http://127.0.0.1:8000/sante" | grep -q '"base_de_donnees":"ok"'; th
        en echo "OK : l'API de $serveur répond et joint la base"; else echo "ERREUR : l'API de $serveur ne répond pas correctement
        (fiche D9)"; fi
done
```

```
...
OK : rejeu sans aucun changement, le playbook est idempotent
OK : l'API de api1 répond et joint la base
OK : l'API de api2 répond et joint la base
```

💬 **Question** : connectez-vous à `api1` (`ssh -i ~/.ssh/tp_iac_ed25519 -p 2242 -o StrictHostKeyChecking=no -o UserKnownHostsFile=/dev/null ansible@127.0.0.1`) et tapez `cat /etc/stockline/stockline.env` . Que se passe-t-il, et pourquoi est-ce voulu ? (`exit` pour revenir.)

Partie 8 — Rôle « nginx » et vérification de bout en bout (40 min)

Le répartiteur `lb1` reçoit les requêtes sur son port 80 (le port **8080** de votre poste) et les distribue **à tour de rôle** entre les API.

Le template `stockline.conf.j2` contient une **boucle Jinja2** sur `groups['api']`, la liste des serveurs du groupe `api` **dans l'inventaire**. La liste des API n'est donc écrite nulle part en dur : elle vient de Terraform, via l'inventaire. L'en-tête de réponse `X-Servi-Par` indique quelle API a répondu.

Les deux handlers s'exécutent **dans l'ordre où ils sont écrits** dans `handlers/main.yml` : d'abord `nginx -t` teste la configuration, ensuite seulement nginx est rechargé. Une configuration cassée arrête le play **avant** le re-chargement, et nginx continue de servir l'ancienne configuration.

```
cd ~/tp-iac/ansible
mkdir -p roles/nginx/tasks roles/nginx/handlers roles/nginx/templates
```

 `~/tp-iac/ansible/roles/nginx/tasks/main.yml`

```
cd ~/tp-iac
cat > ansible/roles/nginx/tasks/main.yml <<'EOF'
# roles/nginx/tasks/main.yml — le répartiteur de charge devant les API
---
- name: Installer nginx
  ansible.builtin.apt:
    name: nginx
    state: present

- name: Déposer la configuration du répartiteur
  ansible.builtin.template:
    src: stockline.conf.j2
    dest: /etc/nginx/sites-available/stockline.conf
    owner: root
    group: root
    mode: "0644"
  notify:
    - Tester la configuration nginx
    - Recharger nginx

- name: Activer le site StockLine
  ansible.builtin.file:
    src: /etc/nginx/sites-available/stockline.conf
    dest: /etc/nginx/sites-enabled/stockline.conf
    state: link
  notify:
    - Tester la configuration nginx
    - Recharger nginx

- name: Désactiver le site par défaut de nginx
  ansible.builtin.file:
    path: /etc/nginx/sites-enabled/default
    state: absent
```

```
  notify:
    - Tester la configuration nginx
    - Recharger nginx

- name: Démarrer nginx et l'activer au démarrage
  ansible.builtin.systemd_service:
    name: nginx
    state: started
    enabled: true

- name: Exécuter maintenant les rechargements en attente
  ansible.builtin.meta: flush_handlers

- name: Vérifier que le répartiteur joint l'API
  ansible.builtin.uri:
    url: http://127.0.0.1/sante
    return_content: true
  register: sante_lb
  until: sante_lb.json is defined and sante_lb.json.base_de_donnees == "ok"
  retries: 10
  delay: 2
EOF
```

 `~/tp-iac/ansible/roles/nginx/handlers/main.yml`

```
cd ~/tp-iac
cat > ansible/roles/nginx/handlers/main.yml <<'EOF'
# roles/nginx/handlers/main.yml
# Les handlers s'exécutent dans l'ordre où ils sont ÉCRITS ici :
# d'abord on teste la configuration, ensuite seulement on recharge.
# Si le test échoue, le play s'arrête et nginx garde l'ancienne configuration.
---
- name: Tester la configuration nginx
  ansible.builtin.command: nginx -t
  changed_when: false

- name: Recharger nginx
  ansible.builtin.systemd_service:
    name: nginx
    state: reloaded
EOF
```

 `~/tp-iac/ansible/roles/nginx/templates/stockline.conf.j2`

```
cd ~/tp-iac
cat > ansible/roles/nginx/templates/stockline.conf.j2 <<'EOF'
# {{ ansible_managed }}
# Répartiteur de charge : les requêtes sont distribuées à tour de rôle
# entre TOUS les serveurs du groupe Ansible « api ».

upstream stockline_api {
{% for serveur in groups['api'] %}
    server {{ hostvars[serveur]['nom_conteneur'] }}:{{ stockline_port }};
{% endfor %}
}

server {
    listen 80 default_server;
    server_name _;
```

```
    location / {
        proxy_pass http://stockline_api;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        # En-tête de réponse indiquant quel serveur api a répondu (IP:port)
        add_header X-Servi-Par $upstream_addr always;
    }
}
EOF
```

`site.yml` dans sa version finale : un play pour `lb`, et un dernier play qui s'exécute **sur votre poste** (`hosts: localhost`, `connection: local`) pour vérifier StockLine comme le ferait un utilisateur, par `http://localhost:8080`.

 `~/tp-iac/ansible/site.yml`

```
cd ~/tp-iac
cat > ansible/site.yml <<'EOF'
# site.yml — LE playbook du projet : chaque play applique des rôles à un groupe
---
- name: Socle commun à tous les serveurs
  hosts: all
  become: true
  roles:
    - commun

- name: Serveur de base de données
  hosts: db
  become: true
  roles:
    - postgresql

- name: Serveurs d'API StockLine (un par un)
  hosts: api
  become: true
  serial: 1
  roles:
    - stockline

- name: Répartiteur de charge
  hosts: lb
  become: true
  roles:
    - nginx

- name: Vérification de bout en bout depuis le poste
  hosts: localhost
  connection: local
  gather_facts: false
  tasks:
    - name: Appeler StockLine via le répartiteur (http://localhost:8080)
      ansible.builtin.uri:
        url: http://localhost:8080/sante
        return_content: true
      register: sante_finale
      until: sante_finale.json is defined and sante_finale.json.base_de_donnees == "ok"
      retries: 10
      delay: 2

    - name: Afficher la réponse de StockLine
      ansible.builtin.debug:
```

```
        var: sante_finale.json
EOF
```

```
cd ~/tp-iac/ansible
ansible-playbook site.yml
```

```
...
TASK [Afficher la réponse de StockLine] ****************************************
ok: [localhost] => {
    "sante_finale.json": {
        "base_de_donnees": "ok",
        "statut": "ok",
        "version": "1.0.0"
    }
}

PLAY RECAP *********************************************************************
api1                       : ok=17   changed=0 ...
api2                       : ok=17   changed=0 ...
db1                        : ok=11   changed=0 ...
lb1                        : ok=13   changed=6 ...
localhost                  : ok=2    changed=0 ...
```

8.1 Utiliser StockLine

Créons un produit, puis un mouvement d'entrée en stock, et lisons le stock, **depuis votre poste**, à travers le répartiteur.

```
cd ~/tp-iac
curl -s http://localhost:8080/sante; echo
curl -s -o /dev/null -w "Création du produit : HTTP %{http_code}\n" -X POST http://localhost:8080/produits -H "Content-Type: appli
        cation/json" -d '{"reference": "SSD-500", "nom": "Disque SSD 500 Go", "prix_unitaire": 59.9, "seuil_alerte": 5}'
curl -s http://localhost:8080/produits; echo
curl -s -o /dev/null -w "Entrée en stock : HTTP %{http_code}\n" -X POST http://localhost:8080/mouvements -H "Content-Type: applica
        tion/json" -d '{"produit_id": 1, "type": "entree", "quantite": 12, "commentaire": "Livraison fournisseur"}'
curl -s http://localhost:8080/stocks/1; echo
```

```
{"statut":"ok","base_de_donnees":"ok","version":"1.0.0"}
Création du produit : HTTP 201
[{"reference":"SSD-500","nom":"Disque SSD 500 Go","prix_unitaire":59.9,"seuil_alerte":5,"id":1,"cree_le":"2026-10-27T15:42:10.75917
8"}]
Entrée en stock : HTTP 201
{"produit_id":1,"reference":"SSD-500","nom":"Disque SSD 500 Go","quantite":12,"seuil_alerte":5,"alerte":false}
```

> Si vous recollez ce bloc, la création renvoie `HTTP 409` : la référence `SSD-500` existe déjà. C'est normal.

Depuis Windows, ouvrez aussi **http://localhost:8080/docs** dans votre navigateur : c'est la documentation interactive de l'API, servie à travers nginx.

8.2 Le répartiteur en action

Les deux API partagent **la même base** : peu importe laquelle répond, les données sont les mêmes. Regardons qui répond à chaque requête :

```
cd ~/tp-iac
for i in 1 2 3 4 5 6; do
  curl -s -o /dev/null -D - http://localhost:8080/sante | grep -i "^x-servi-par"
done
docker network inspect tp-iac-net --format '{{range .Containers}}{{.Name}} -> {{.IPv4Address}}{{println}}{{end}}'
```

```
X-Servi-Par: 172.18.0.4:8000
X-Servi-Par: 172.18.0.2:8000
X-Servi-Par: 172.18.0.4:8000
X-Servi-Par: 172.18.0.2:8000
X-Servi-Par: 172.18.0.4:8000
X-Servi-Par: 172.18.0.2:8000
tp-iac-api2 -> 172.18.0.2/16
tp-iac-db1 -> 172.18.0.5/16
tp-iac-lb1 -> 172.18.0.3/16
tp-iac-api1 -> 172.18.0.4/16
```

Les adresses alternent : une requête sur deux part vers `api1`, l'autre vers `api2` (vos adresses IP seront différentes).

✅ **Point de contrôle 8** :

```
cd ~/tp-iac/ansible
ansible-playbook site.yml | tee /tmp/tp-iac-rejeu.log
if grep -Eq "changed=[1-9]|failed=[1-9]|unreachable=[1-9]" /tmp/tp-iac-rejeu.log; then echo "ERREUR : le rejeu a encore changé que
        lque chose (relisez le PLAY RECAP)"; else echo "OK : rejeu sans aucun changement, le playbook est idempotent"; fi
nb=$(for i in 1 2 3 4 5 6; do curl -s -o /dev/null -D - http://localhost:8080/sante | grep -i "^x-servi-par"; done | sort -u | wc
        -l)
if [ "$nb" -eq 2 ]; then echo "OK : le répartiteur distribue les requêtes entre 2 API"; else echo "ERREUR : $nb API différente(s)
        ont répondu au lieu de 2"; fi
```

```
...
OK : rejeu sans aucun changement, le playbook est idempotent
OK : le répartiteur distribue les requêtes entre 2 API
```

Partie 9 — Passer à l'échelle : un 3e serveur d'API (35 min)

La charge augmente : il faut une troisième API. Sans infrastructure as code, il faudrait créer le conteneur, recopier son port dans l'inventaire, l'installer, et modifier la configuration nginx. Ici, **on ajoute une ligne** dans `variables.tf`, celle d'`api3` :

`~/tp-iac/terraform/variables.tf`

```
cd ~/tp-iac
cat > terraform/variables.tf <<'EOF'
# variables.tf — les paramètres du projet
variable "prefixe" {
  type        = string
  description = "Préfixe de tous les noms Docker créés par ce projet"
  default     = "tp-iac"
}

variable "cle_ssh_privee" {
  type        = string
  description = "Clé privée qu'Ansible utilisera pour se connecter aux serveurs"
  default     = "~/.ssh/tp_iac_ed25519"
}

# Le parc de serveurs : une entrée = un conteneur.
#   groupe    : groupe Ansible du serveur (db, api ou lb)
#   port_ssh  : port du poste redirigé vers son SSH
#   port_http : port du poste redirigé vers son port 80 (facultatif)
variable "serveurs" {
  type = map(object({
    groupe    = string
    port_ssh  = number
    port_http = optional(number)
  }))
  description = "Serveurs à créer, indexés par leur nom court"

  default = {
    db1  = { groupe = "db", port_ssh = 2241 }
    api1 = { groupe = "api", port_ssh = 2242 }
    api2 = { groupe = "api", port_ssh = 2243 }
    api3 = { groupe = "api", port_ssh = 2245 }
    lb1  = { groupe = "lb", port_ssh = 2244, port_http = 8080 }
  }
}
EOF
```

Lisez le plan **avant** d'appliquer :

```
cd ~/tp-iac/terraform
terraform plan -out=tfplan
```

```
  # local_file.inventaire must be replaced
-/+ resource "local_file" "inventaire" {
      ~ content              = <<-EOT
            ...
            api2 ansible_host=127.0.0.1 ansible_port=2243 nom_conteneur=tp-iac-api2
          + api3 ansible_host=127.0.0.1 ansible_port=2245 nom_conteneur=tp-iac-api3
            ...
        EOT # forces replacement
      ...
    }

  # module.serveur["api3"].docker_container.serveur will be created
  + resource "docker_container" "serveur" {
      + name = "tp-iac-api3"
```

```
      ...
    }

Plan: 2 to add, 0 to change, 1 to destroy.
```

Le plan est exactement celui qu'on veut : **un** conteneur créé, l'inventaire **réécrit** avec une ligne de plus, et **aucun** des 4 serveurs existants touché. Le « 1 to destroy » est l'ancien fichier d'inventaire, remplacé par le nouveau.

```
cd ~/tp-iac/terraform
terraform apply tfplan
grep api3 ~/tp-iac/ansible/inventory.ini
```

```
Apply complete! Resources: 2 added, 0 changed, 1 destroyed.
...
api3 ansible_host=127.0.0.1 ansible_port=2245 nom_conteneur=tp-iac-api3
```

On relance **le même playbook, sans y toucher** :

```
cd ~/tp-iac/ansible
sleep 5
ansible-playbook site.yml
```

Dans le PLAY RECAP, seuls `api3` (installé entièrement) et `lb1` (sa configuration nginx gagne une ligne `server tp-iac-api3:8000;`, puis le handler recharge nginx) ont des `changed` ; les autres serveurs sont à `changed=0`. Comptez environ 2 minutes.

```
PLAY RECAP *********************************************************************
api1                       : ok=17   changed=0 ...
api2                       : ok=17   changed=0 ...
api3                       : ok=18   changed=14 ...
db1                        : ok=11   changed=0 ...
lb1                        : ok=13   changed=2 ...
localhost                  : ok=2    changed=0 ...
```

✅ **Point de contrôle 9** :

```
cd ~/tp-iac/ansible
ansible lb1 -m ansible.builtin.command -a "cat /etc/nginx/sites-available/stockline.conf" | grep "server tp-iac"
nb=$(for i in 1 2 3 4 5 6 7 8 9; do curl -s -o /dev/null -D - http://localhost:8080/sante | grep -i "^x-servi-par"; done | sort -u
        | wc -l)
if [ "$nb" -eq 3 ]; then echo "OK : le répartiteur distribue les requêtes entre 3 API"; else echo "ERREUR : $nb API différente(s)
        ont répondu au lieu de 3"; fi
if curl -s http://localhost:8080/produits | grep -q "SSD-500"; then echo "OK : les données créées en partie 8 sont toujours là"; e
        lse echo "ERREUR : le produit SSD-500 a disparu"; fi
```

```
    server tp-iac-api1:8000;
    server tp-iac-api2:8000;
    server tp-iac-api3:8000;
```

```
OK : le répartiteur distribue les requêtes entre 3 API
OK : les données créées en partie 8 sont toujours là
```

> 💬 **Question** : combien de fichiers avez-vous modifiés pour ajouter un serveur ? Faites la liste de tout ce que Terraform puis Ansible ont fait à votre place.

Partie 10 — Tout détruire (15 min)

Fin de journée : on ne laisse rien tourner. Même discipline que pour `apply` : on **planifie** la destruction, on la **lit**, puis on applique ce plan.

```
cd ~/tp-iac/terraform
terraform plan -destroy -out=tfplan-destroy
terraform apply tfplan-destroy
```

```
Plan: 0 to add, 0 to change, 8 to destroy.
...
Apply complete! Resources: 0 added, 0 changed, 8 destroyed.
```

Les **8** objets détruits : 5 conteneurs, l'image, le réseau et le fichier d'inventaire.

✅ **Point de contrôle 10** :

```
cd ~/tp-iac/terraform
if [ -z "$(docker ps -aq --filter name=tp-iac-)" ]; then echo "OK : plus aucun conteneur tp-iac"; else echo "ERREUR : il reste des
        conteneurs tp-iac (fiche D10)"; fi
if docker image inspect tp-iac-cible:1.0 > /dev/null 2>&1; then echo "ERREUR : l'image tp-iac-cible:1.0 existe encore (fiche D1
        0)"; else echo "OK : plus d'image tp-iac-cible"; fi
if docker network inspect tp-iac-net > /dev/null 2>&1; then echo "ERREUR : le réseau tp-iac-net existe encore (fiche D10)"; else e
        cho "OK : plus de réseau tp-iac-net"; fi
if [ -e ~/tp-iac/ansible/inventory.ini ]; then echo "ERREUR : l'inventaire généré existe encore"; else echo "OK : l'inventaire gén
        éré a été supprimé"; fi
if [ -z "$(terraform state list)" ]; then echo "OK : le state Terraform est vide"; else echo "ERREUR : le state Terraform n'est pa
        s vide"; fi
```

```
OK : plus aucun conteneur tp-iac
OK : plus d'image tp-iac-cible
OK : plus de réseau tp-iac-net
OK : l'inventaire généré a été supprimé
OK : le state Terraform est vide
```

Gardez le dossier `~/tp-iac` : c'est votre code. Tout l'environnement peut être reconstruit à l'identique à partir de lui (voir le bonus).

Bonus — Tout reconstruire en deux commandes

Si vous avez du temps : la preuve ultime de l'infrastructure as code. À partir du seul code, on reconstruit **tout** l'environnement (5 serveurs, base, API, répartiteur) puis on le détruit à nouveau. Chronométrez-vous.

```
cd ~/tp-iac/terraform
time terraform apply -auto-approve
cd ~/tp-iac/ansible
sleep 5
time ansible-playbook site.yml
curl -s http://localhost:8080/sante; echo
```

```
...
Apply complete! Resources: 8 added, 0 changed, 0 destroyed.
...
{"statut":"ok","base_de_donnees":"ok","version":"1.0.0"}
```

Le produit `SSD-500` a disparu : la base a été recréée vide. Les **données** ne sont pas de l'infrastructure : en production, elles se sauvegardent à part (sauvegardes, volumes persistants).

`-auto-approve` applique sans montrer de plan ni demander de confirmation : à réserver aux reconstructions dont on connaît exactement le contenu, jamais à un premier `apply`. Pour finir, on détruit à nouveau :

```
cd ~/tp-iac/terraform
terraform plan -destroy -out=tfplan-destroy
terraform apply tfplan-destroy
if [ -z "$(docker ps -aq --filter name=tp-iac-)" ]; then echo "OK : plus aucun conteneur tp-iac"; else echo "ERREUR : il reste des
        conteneurs tp-iac (fiche D10)"; fi
```

Ce qu'il faut retenir

- **Terraform possède ce qui existe** (image, réseau, conteneurs, et même le fichier d'inventaire) ; **Ansible possède ce qu'il y a dedans**.
- **Une seule source de vérité** : la variable `serveurs`. L'inventaire, la configuration nginx et la connexion à la base en découlent ; rien n'est recopié à la main.
- **Toujours lire le plan** (`plan -out`, puis `apply` de ce plan), y compris pour détruire.
- **Idempotence** : un playbook décrit un état ; un rejeu sur un parc conforme affiche `changed=0`.
- **Handlers** : on redémarre ou recharge un service **seulement** quand sa configuration a changé, et on **teste** avant de recharger.

- **Secrets** : chiffrés dans le coffre, référencés par indirection, fichiers d'environnement en `0640`, `no_log` et `diff: false`.
- **Le teardown fait partie du travail.**

Dépannage

D1 — `permission denied … /var/run/docker.sock` : votre utilisateur n'est pas (encore) dans le groupe `docker`.

```
sudo usermod -aG docker $USER
```

Puis fermez **toutes** les fenêtres Ubuntu, tapez `wsl --shutdown` dans PowerShell, rouvrez Ubuntu et relancez la partie 0.

D2 — Terraform ou Ansible absent : reprenez les blocs « Installer Terraform » et « Installer Ansible » du fichier `00-WSL/commands.sh` du cours, puis relancez la partie 0.

D3 — collection `community.postgresql` absente (Ansible installé en `ansible-core` seul) :

```
ansible-galaxy collection install community.postgresql
```

D4 — port déjà utilisé : identifiez qui l'occupe.

```
docker ps --format "table {{.Names}}\t{{.Ports}}"
```

S'il s'agit de conteneurs `tp-iac-…` d'une tentative précédente, supprimez-les avec `docker rm -f $(docker ps -aq --filter name=tp-iac-)`. S'il s'agit d'autres conteneurs d'un exercice précédent, arrêtez-les avec `docker stop <nom>`.

D5 — `~/tp-iac` existe déjà : mettez l'ancienne tentative de côté, sans la supprimer.

```
mv ~/tp-iac ~/tp-iac.ancien-$(date +%Hh%M)
```

D6 — SSH refusé (`Permission denied (publickey)`) : l'image a été construite avec une autre clé que `~/.ssh/tp_iac_ed25519`. Recopiez la bonne clé publique, puis replanifiez : grâce au `triggers` de l'image, Terraform reconstruit l'image et recrée les conteneurs.

```
cp ~/.ssh/tp_iac_ed25519.pub ~/tp-iac/image-cible/authorized_keys
cd ~/tp-iac/terraform && terraform plan -out=tfplan && terraform apply tfplan
```

Si l'erreur est `Connection refused`, le conteneur démarre encore : attendez 10 secondes et relancez le contrôle.

D7 — Ansible : `UNREACHABLE` : vérifiez que les conteneurs tournent (`docker ps`). Après un redémarrage de Windows, ils redémarrent seuls (`restart = "unless-stopped"`) ; sinon relancez `terraform apply` depuis `~/tp-iac/terraform`.

D8 — Ansible ignore `ansible.cfg` (`[WARNING]: Ansible is being run in a world writable directory`) : vous travaillez sous `/mnt/c/…`. Le projet doit être dans `~/tp-iac` (système de fichiers Linux).

D9 — `/sante` répond `"base_de_donnees":"indisponible"` : l'API ne joint pas PostgreSQL. Rejouez d'abord le playbook complet (`ansible-playbook site.yml` depuis `~/tp-iac/ansible`), puis regardez les journaux de l'API :

```
ansible api1 -b -m ansible.builtin.command -a "journalctl -u stockline -n 30 --no-pager"
```

D10 — il reste des objets après le destroy (conteneurs créés à la main, ou state perdu) :

```
docker rm -f $(docker ps -aq --filter name=tp-iac-)
docker image rm tp-iac-cible:1.0
docker network rm tp-iac-net
```

D11 — `http://localhost:8080` ne répond pas dans le navigateur Windows alors que `curl` fonctionne dans Ubuntu : utilisez l'adresse IP de WSL, affichée par `hostname -I` (premier nombre), par exemple `http://172.24.80.5:8080/docs`.

D12 — Un bloc `cat > … <<'EOF'` ne se termine pas (le terminal affiche `>` et attend) : la dernière ligne `EOF` n'a pas été collée. Tapez `EOF` puis Entrée, et recollez le bloc **en entier**.